Experiment No: 03

Working with Amazon DynamoDB

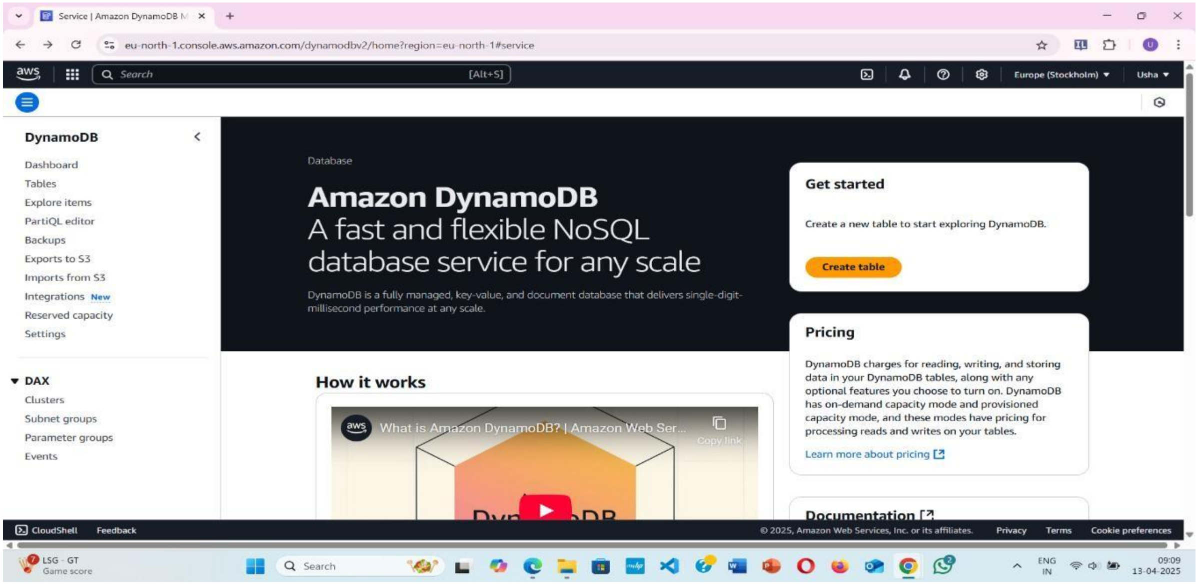
Apparatus Required: AWS Account, AWS Dynamo DB

Pre-Requisite: Aws basics, DBMS basics

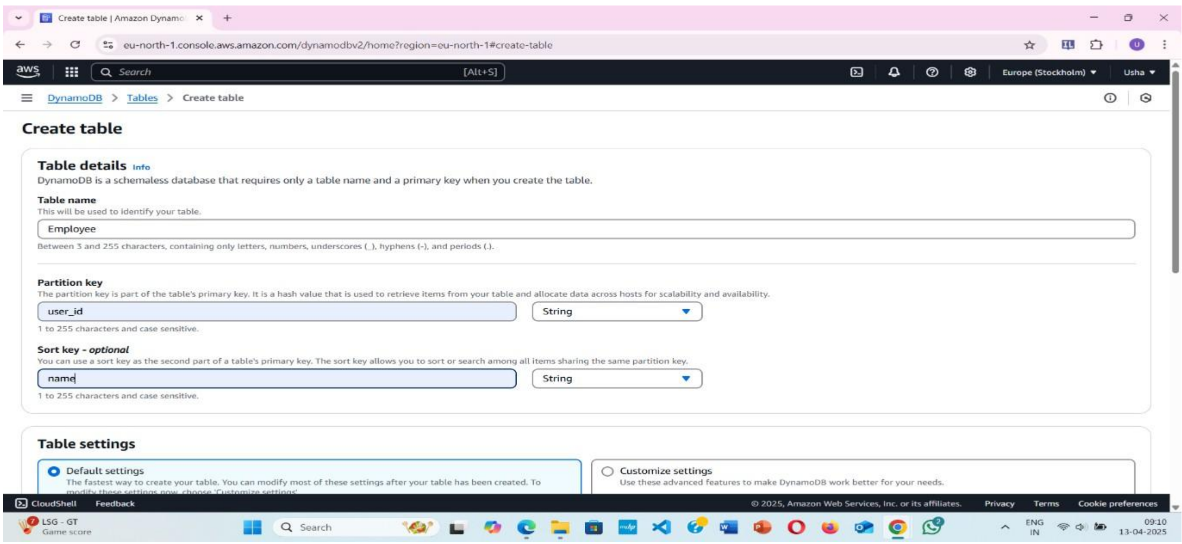
Introduction: AWS DynamoDB is a fast and flexible NoSQL database service for all applications that need consistent, single-digit-millisecond latency at any scale. Its flexible data model and reliable performance make DynamoDB a great fit for mobile, web, gaming, advertising technology, Internet of Things, and other applications.

Procedure

Navigate to Amazon DynamoDB→click on create table



Give table name, Partition Key (is a primary key)



Successfully created the table

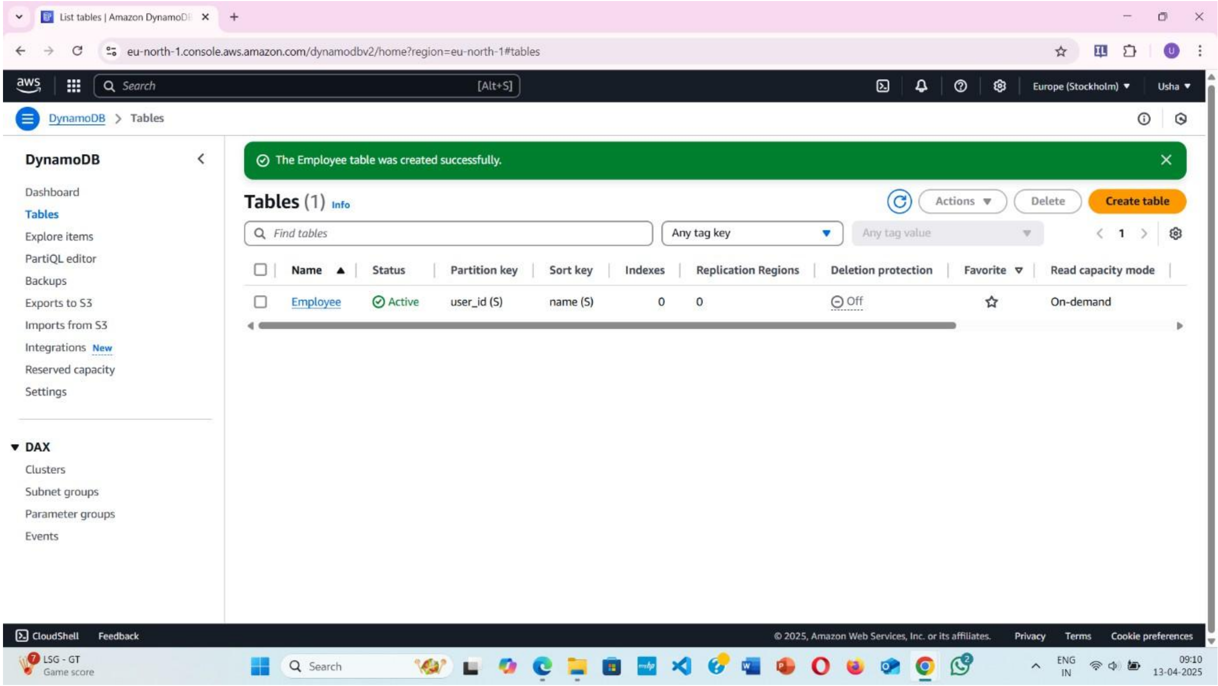
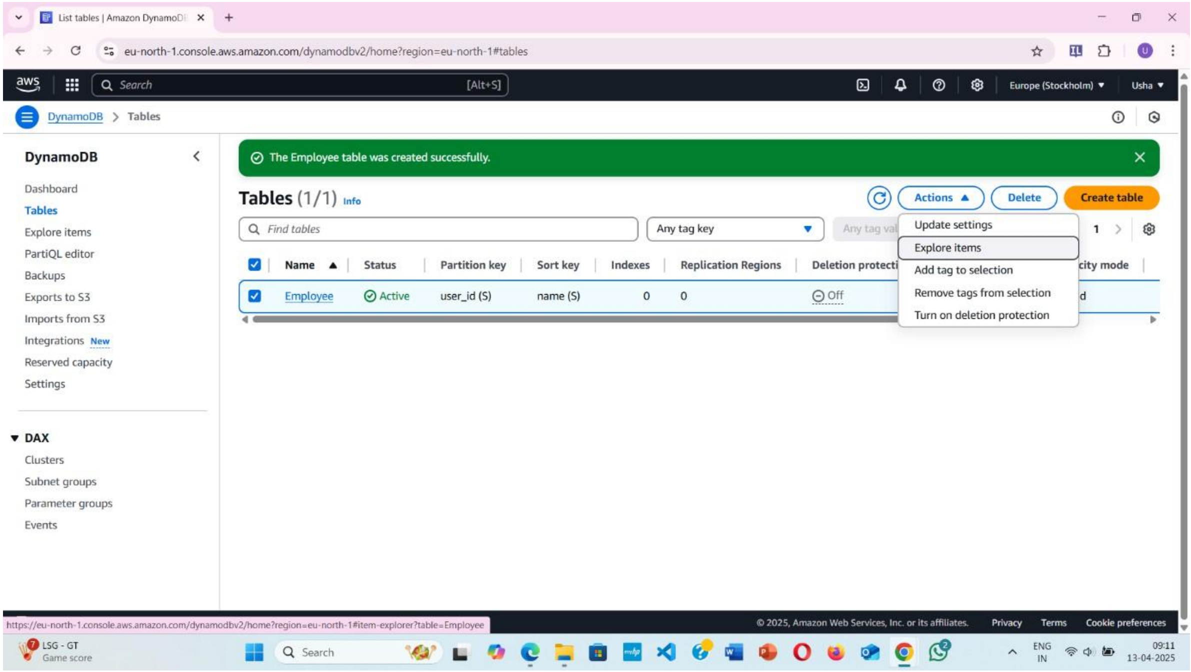
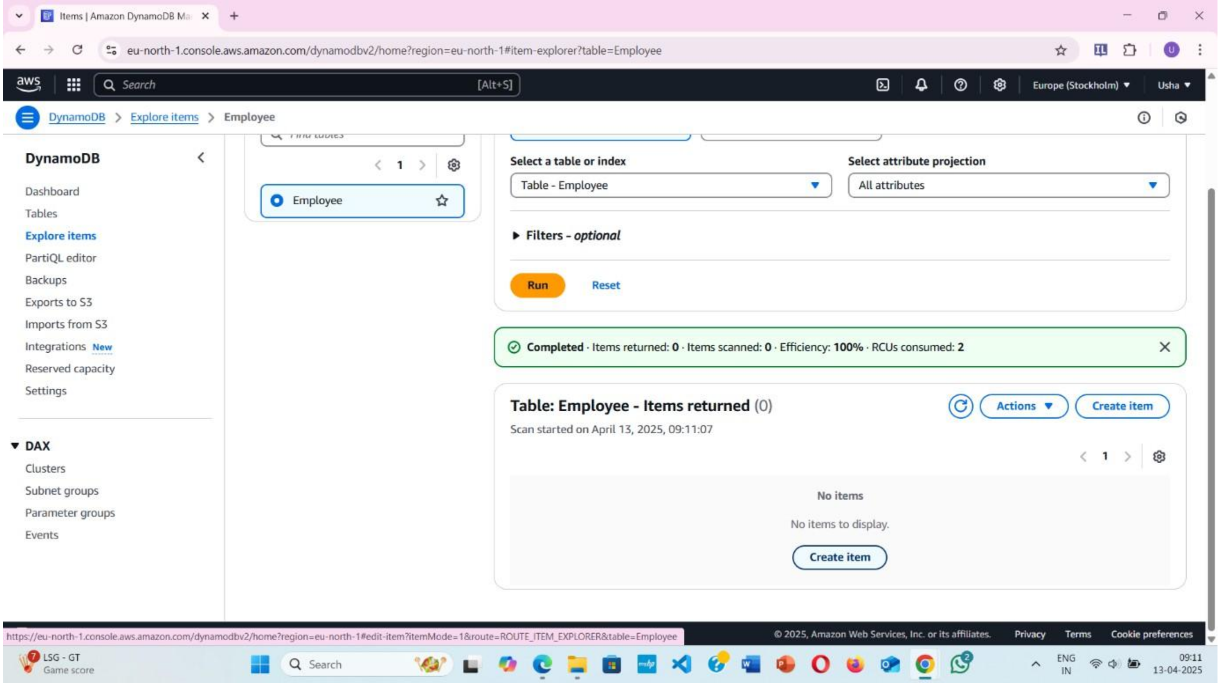
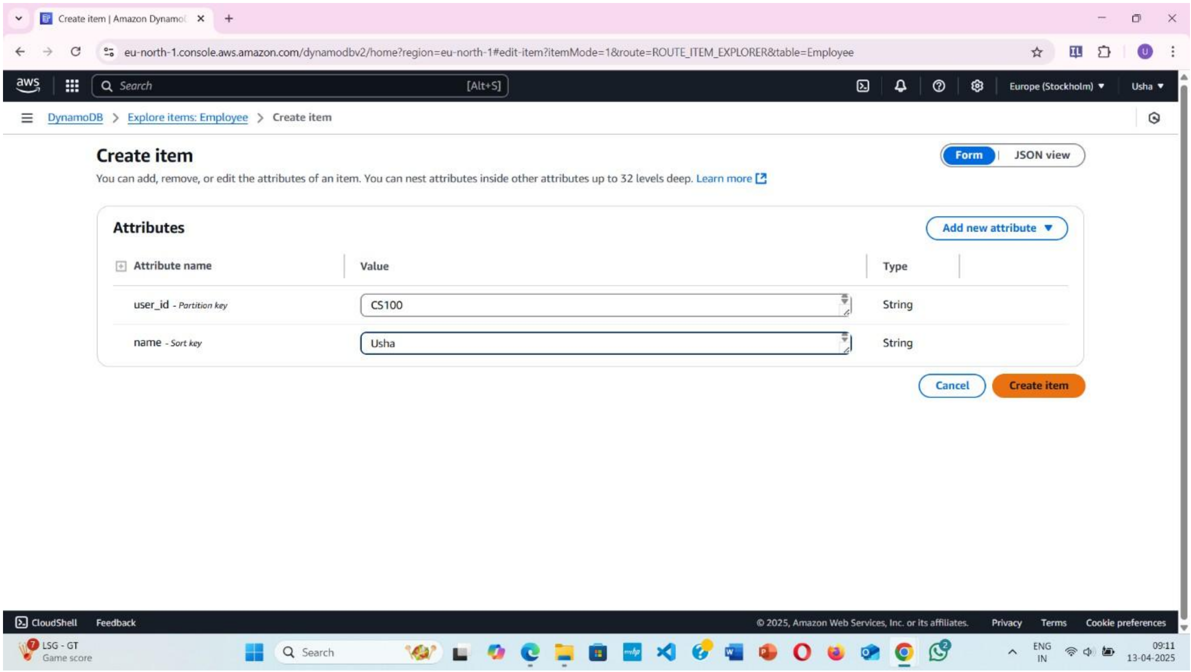
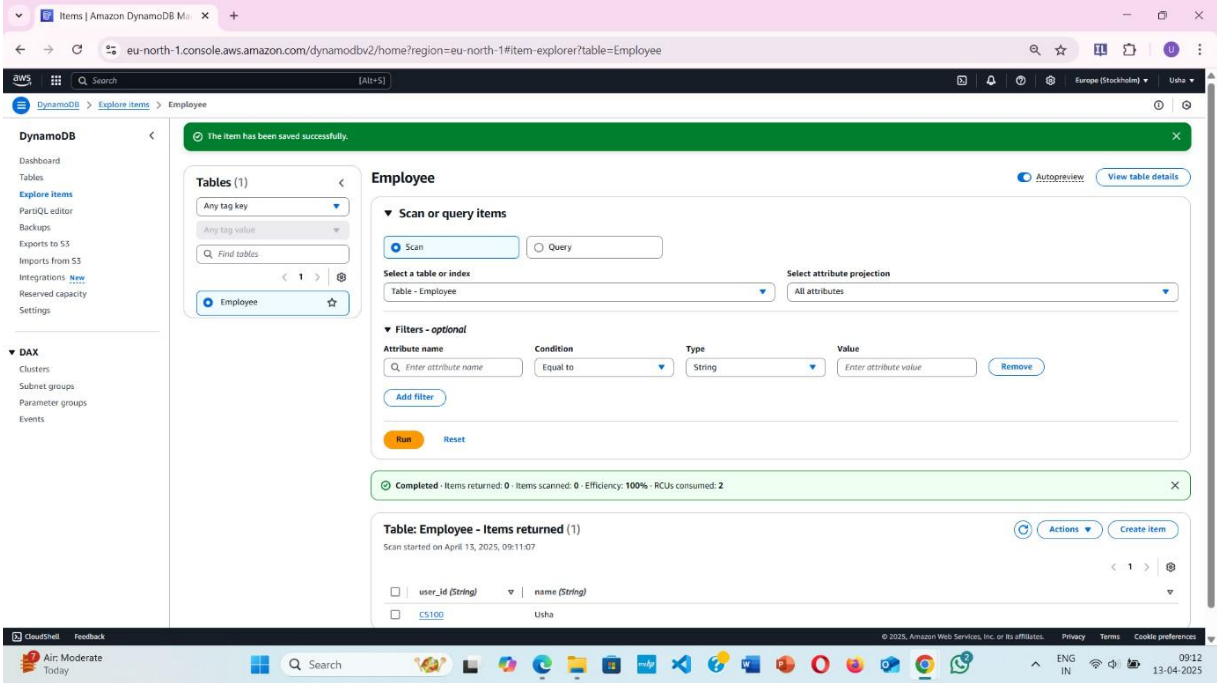


Table created→ select your table→ explore table items→ create items

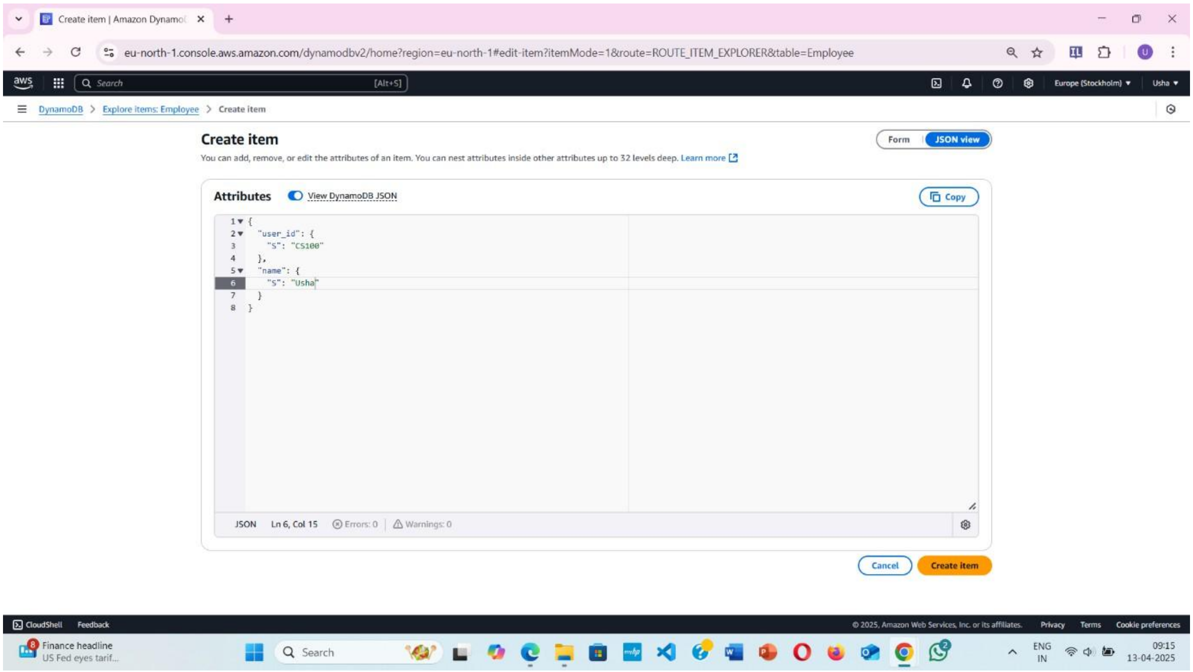


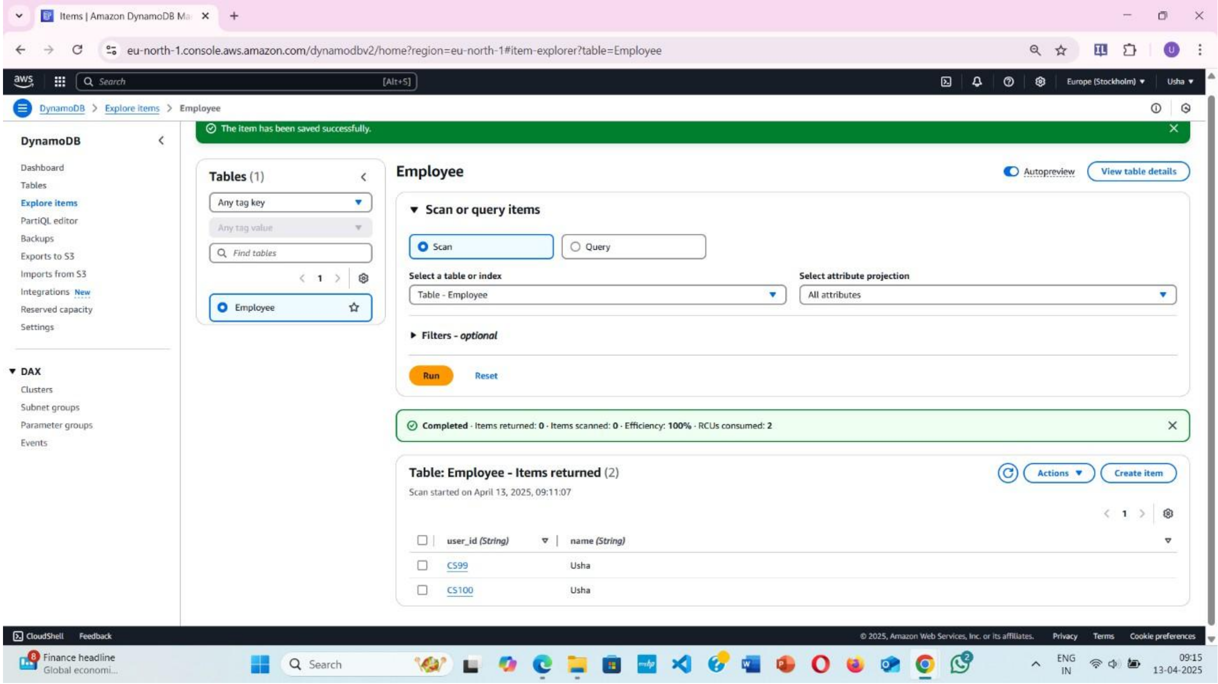

Items | Amazon DynamoDB Ma
eu-north-1.console.aws.amazon.com/dynamodbv2/home?region=eu-north-1#item-explorer?table=Employee
DynamoDB > Explore items > Employee
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations New
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Employee
Select a table or index
Table - Employee
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 0 · Items scanned: 0 · Efficiency: 100% · RCUs consumed: 2
Table: Employee - Items returned (0)
Scan started on April 13, 2025, 09:11:07
Actions
Create item
No items
No items to display.
Create item


Create item | Amazon Dynamo
eu-north-1.console.aws.amazon.com/dynamodbv2/home?region=eu-north-1#edit-item?itemMode=1&route=ROUTE_ITEM_EXPLORER&table=Employee
DynamoDB > Explore items: Employee > Create item
Create item
Form
JSON view
You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. Learn more
Attributes
Add new attribute
Attribute name
Value
Type
user_id - Partition key
CS100
String
name - Sort key
Usha
String
Cancel
Create item



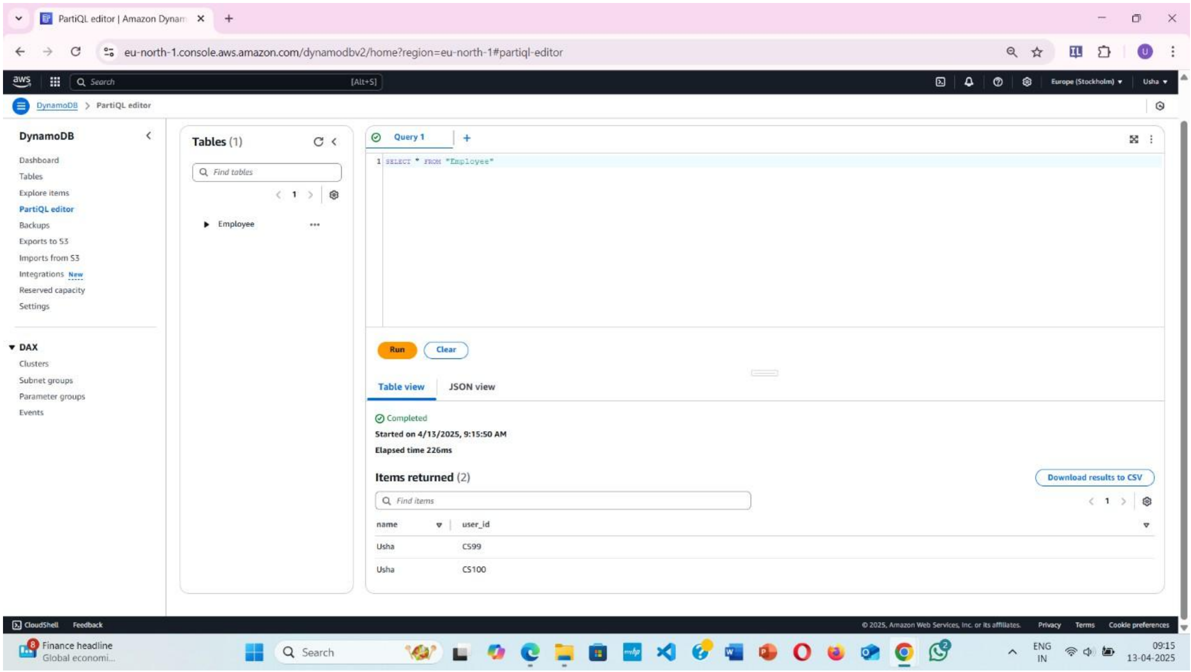
Using JSON to create items



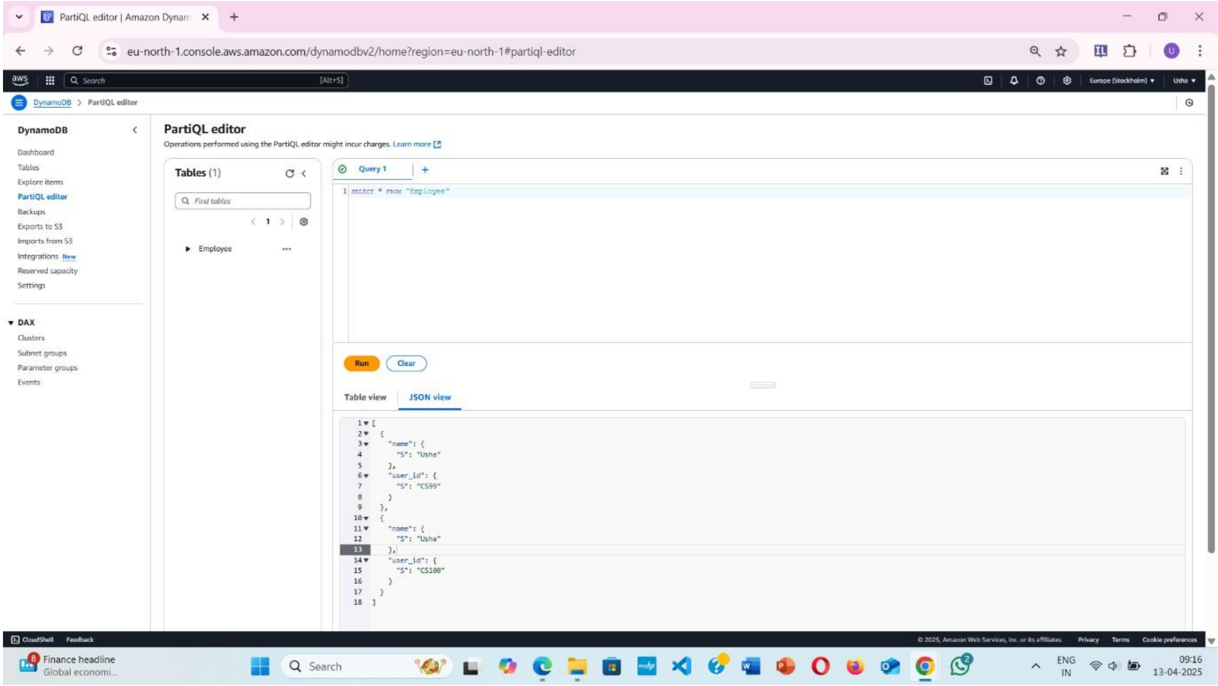


DynamoDB→ PartiQL editor→ write the queries

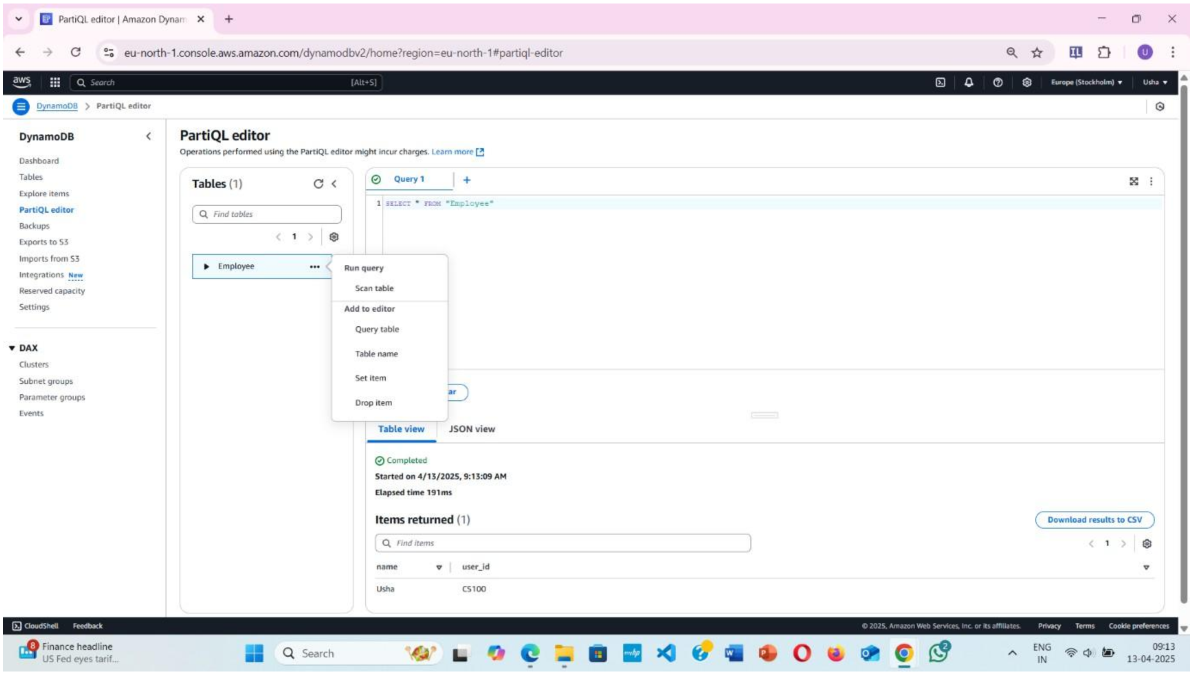
Table view



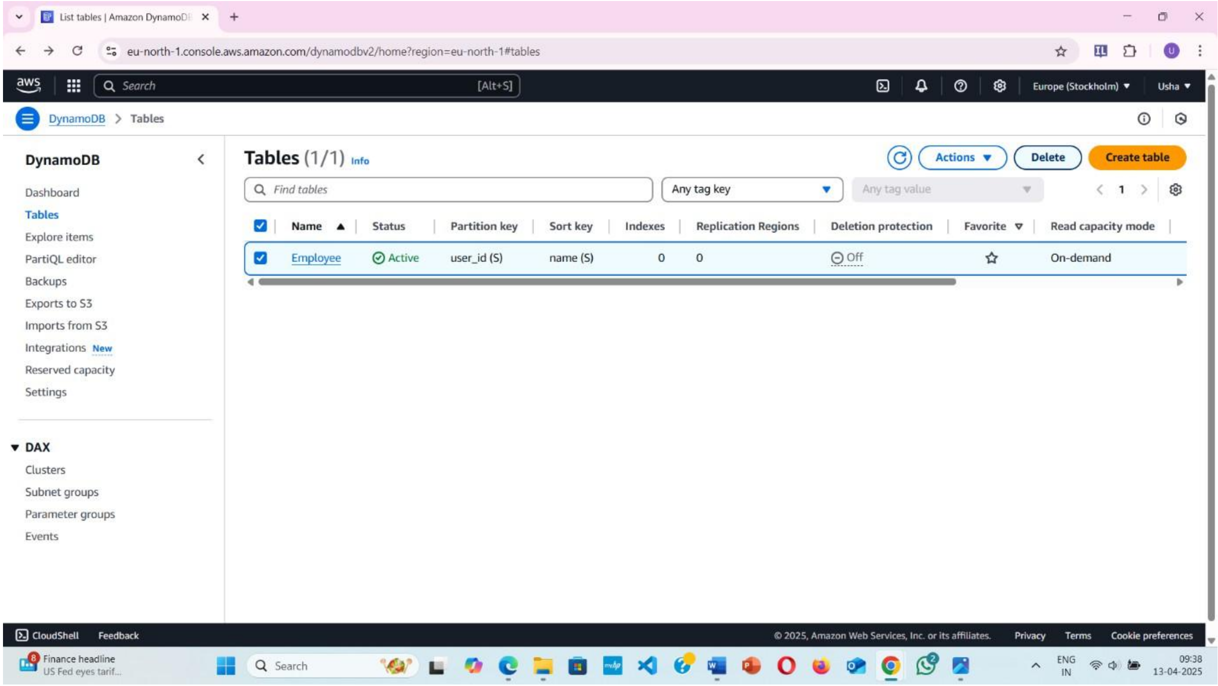
JSON view

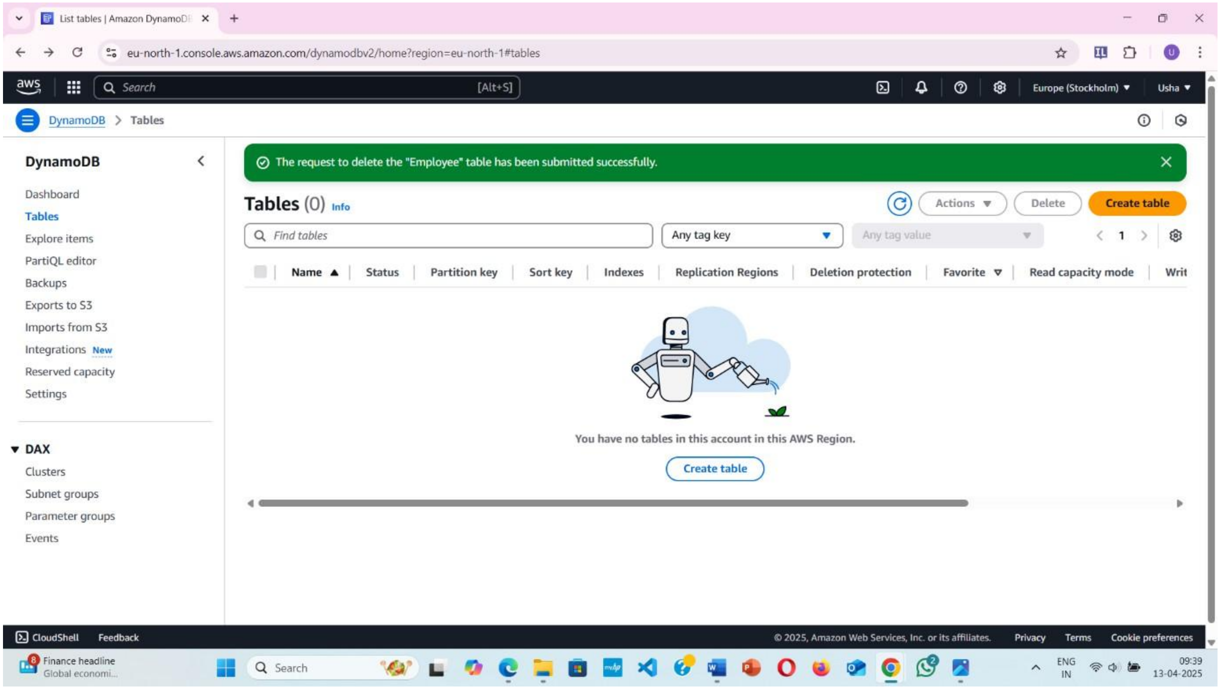


Update Command



Delete table





Observations:

Tables and items are created, and we can run the commands to manipulate the data in the tables.

Experiment No – 04

Developing REST APIs with Amazon API Gateway

Objectives: Developing REST APIs with Amazon API Gateway.

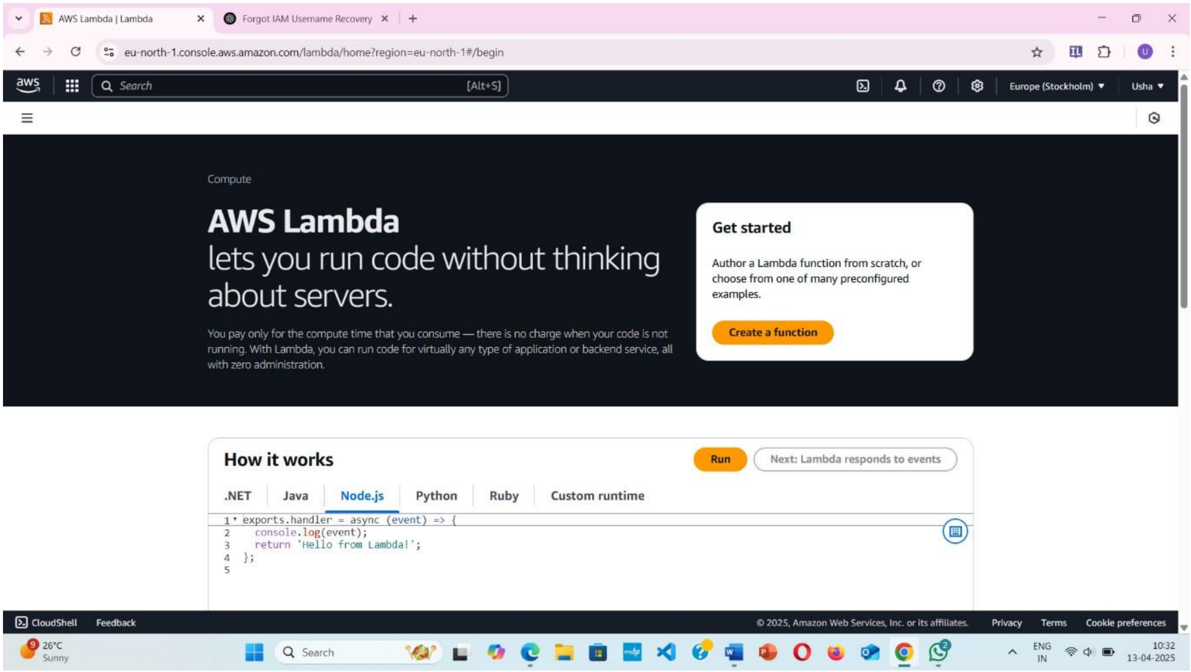
Apparatus Required: AWS Account

Pre-Requisite: Pre - requisite is creation of Lambda Function

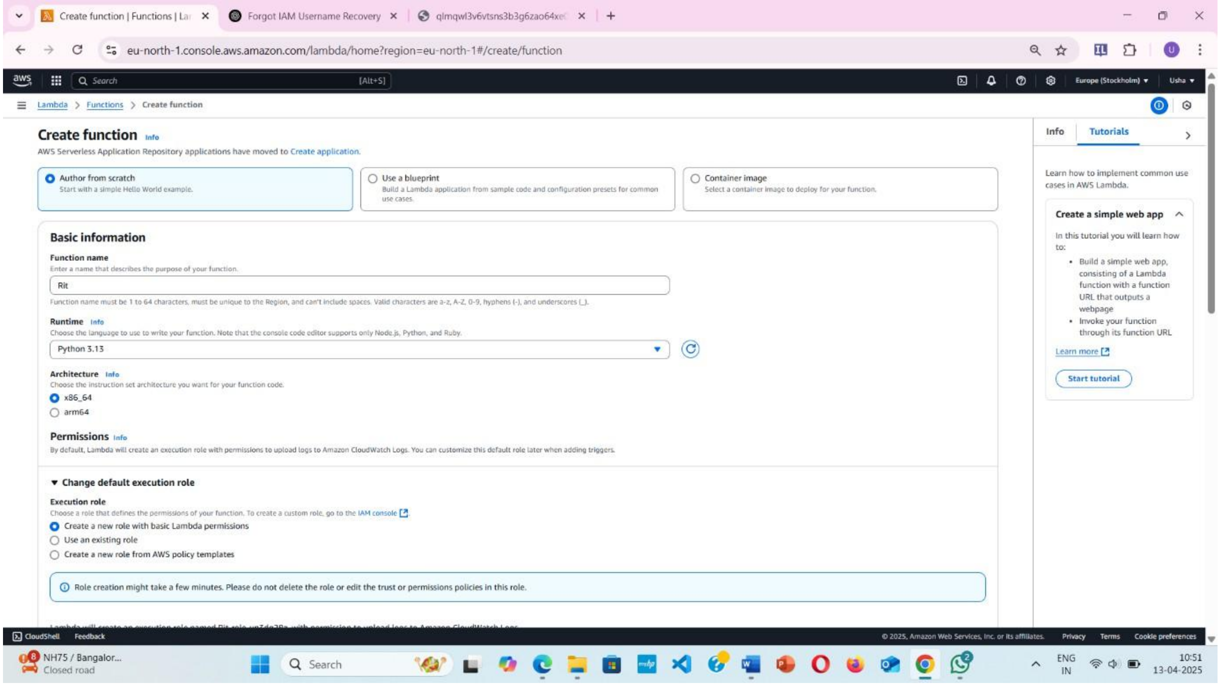
Introduction: API Gateway enables you to connect and access data, business logic, and functionality from backend services such as workloads running on Amazon Elastic Compute Cloud (Amazon EC2), code running on AWS Lambda, any web application, or real-time communication applications.

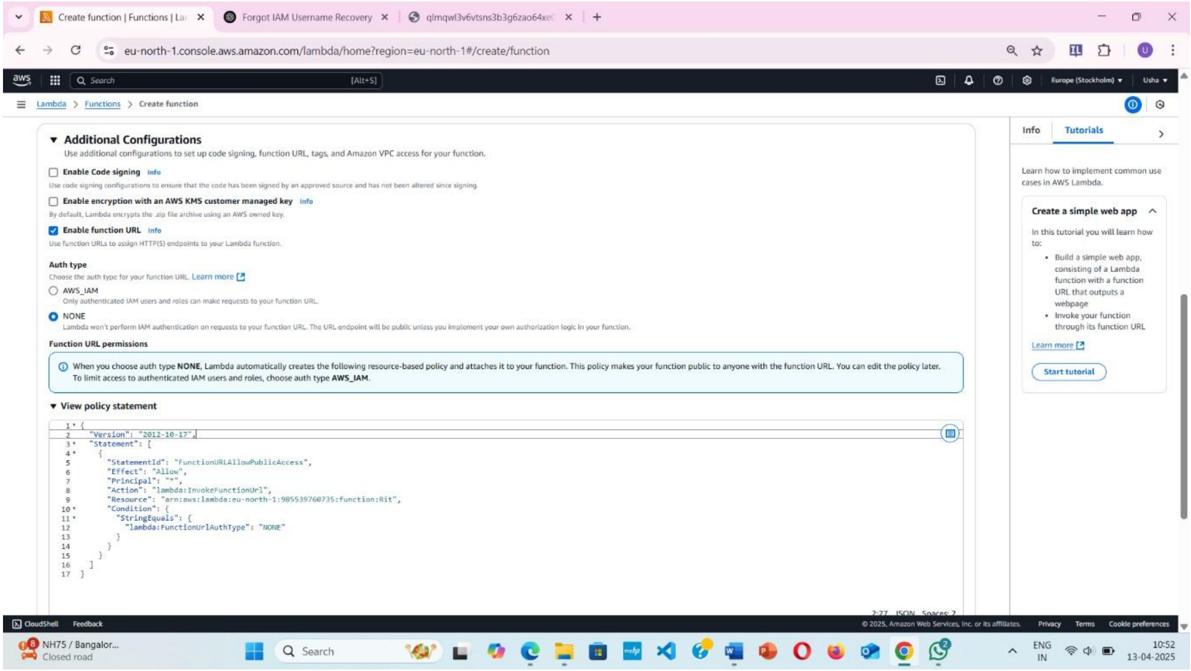
Procedure

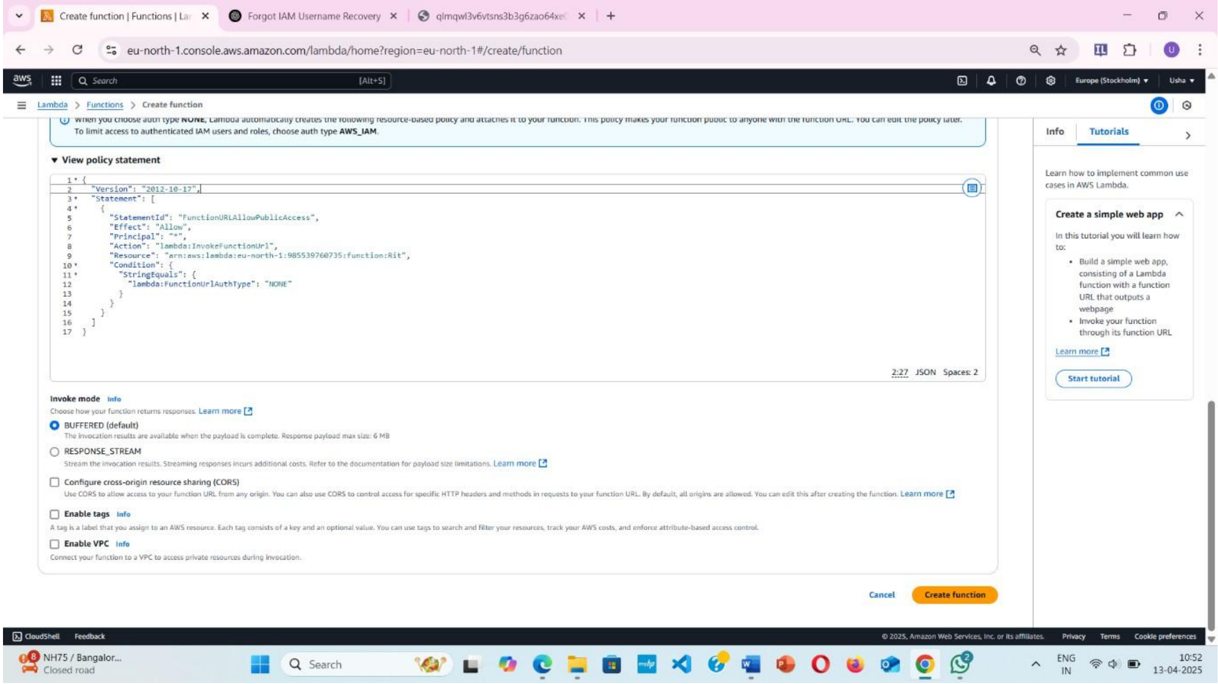
Open AWS Lambda



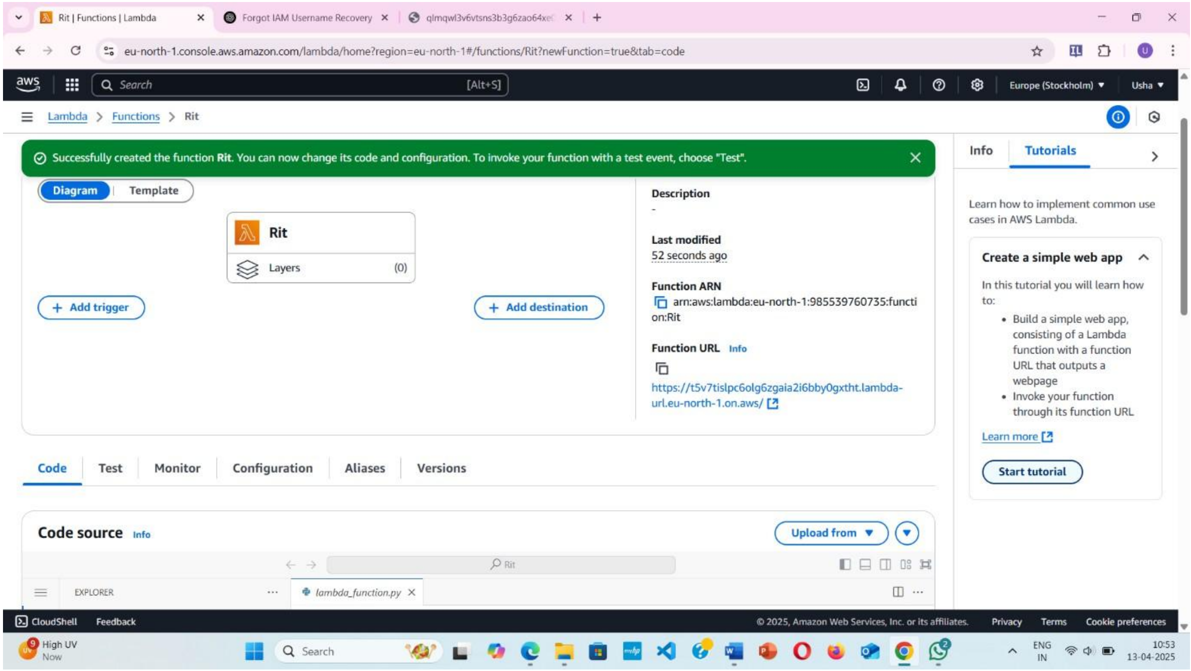
Create function

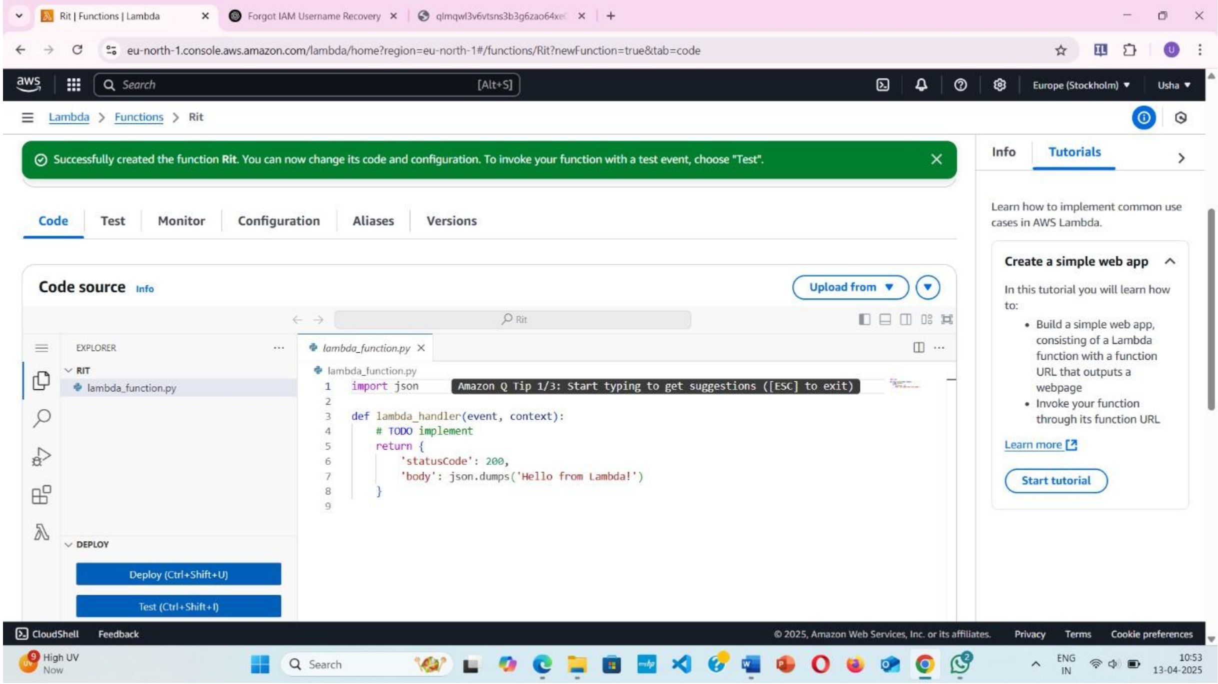




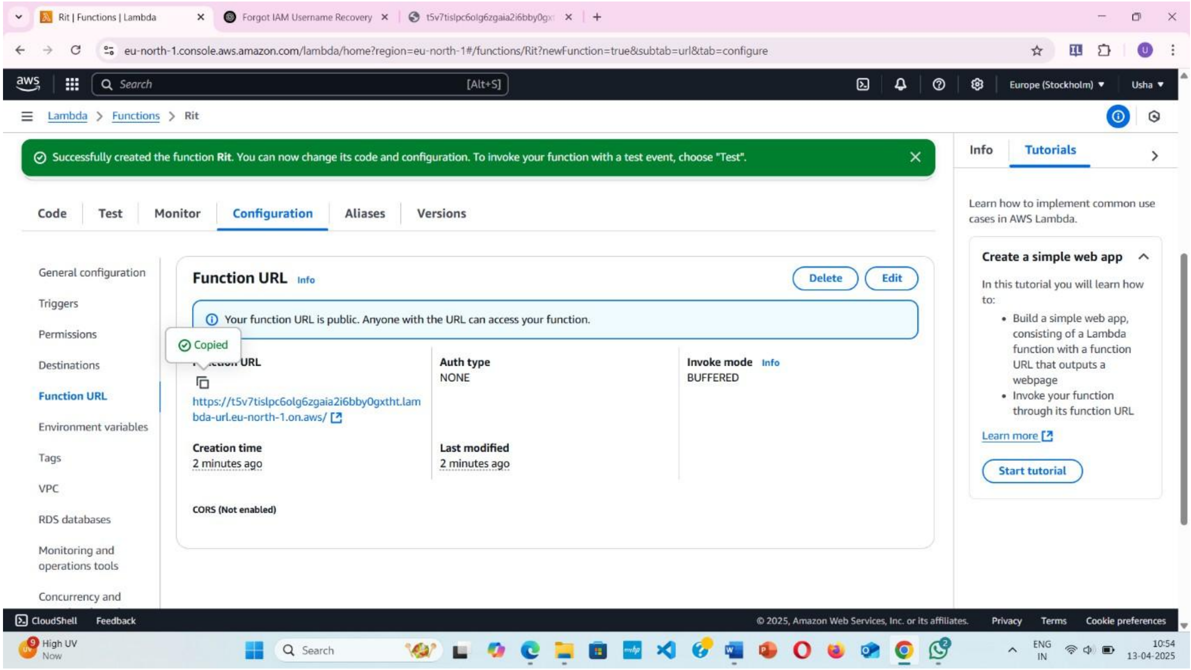


Successfully created the function





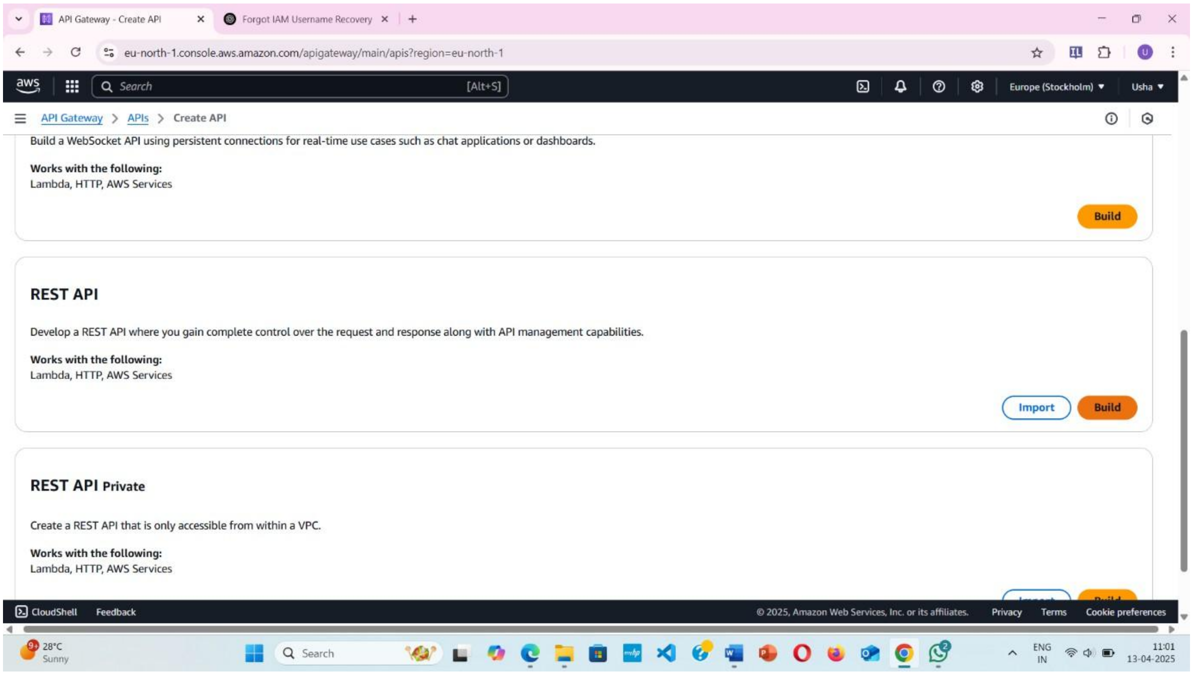
Configuration→ function URL→check URL runs



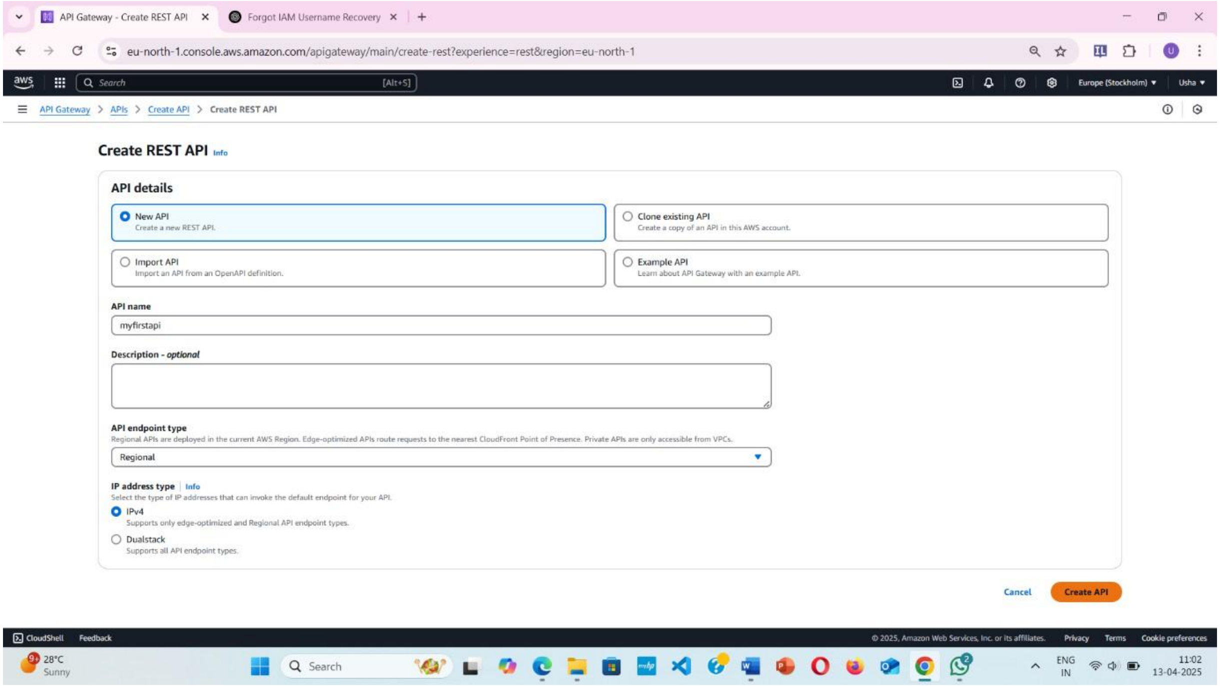
Lambda Code Samples

```
import json
print(json.dumps({"name": "John", "age": 30}))
print(json.dumps(["apple", "banan as"]))
print(json.dumps(("apple", "bananas")))
print(json.dumps("hello"))
print(json.dumps(42))
print(json.dumps(31.76))
print(json.dumps(True))
print(json.dumps(False))
print(json.dumps(None))
```

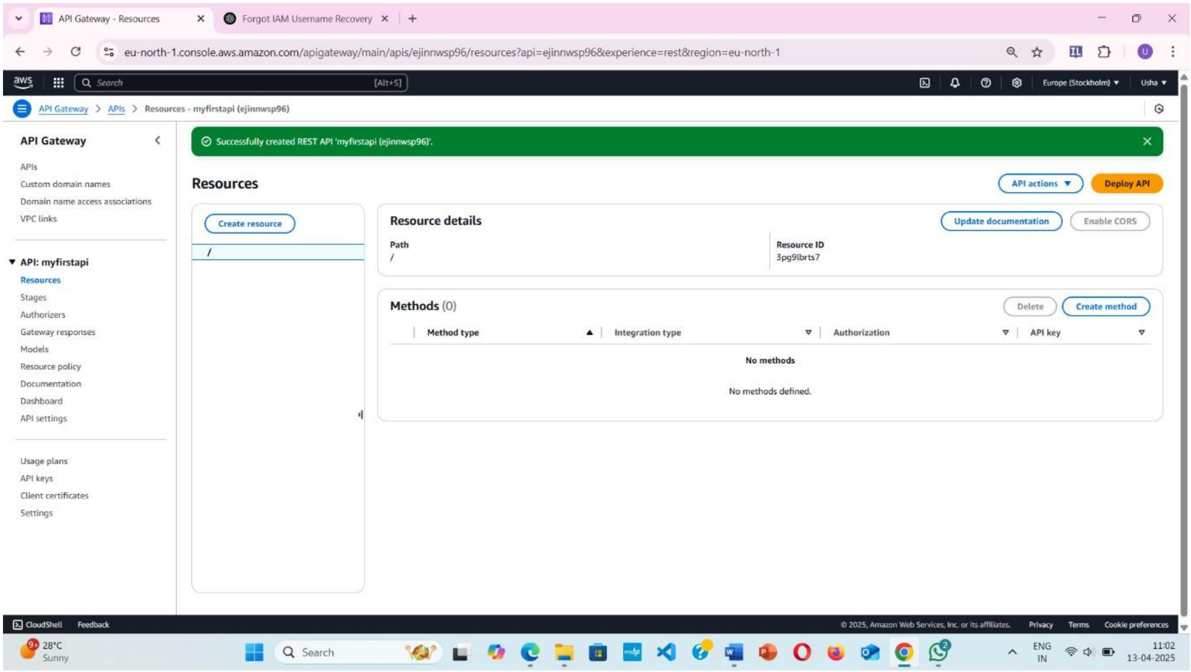
Go to API Gateway in Console And Select Rest API



Build Rest API

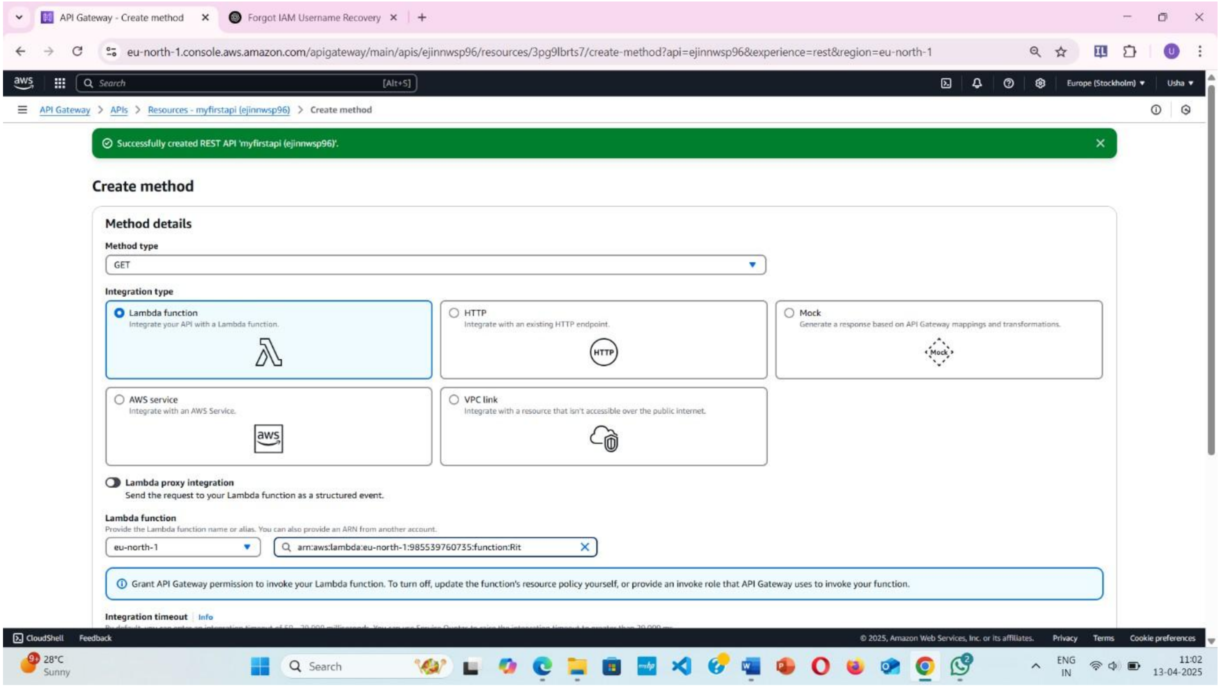


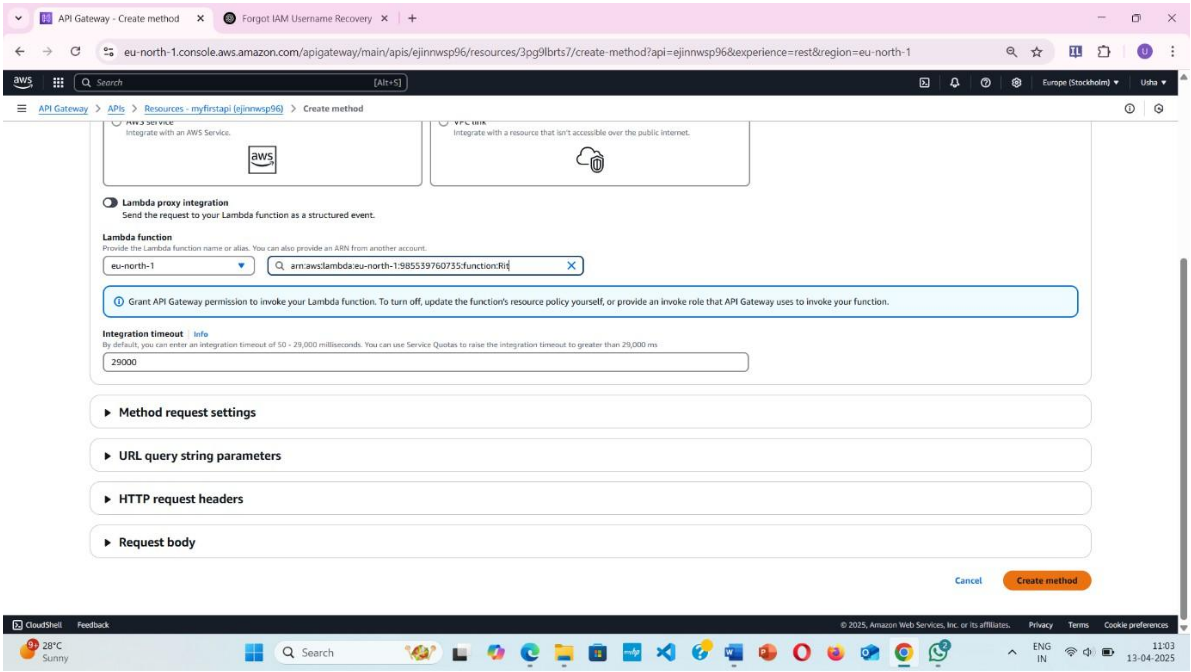
Successfully created the REST API



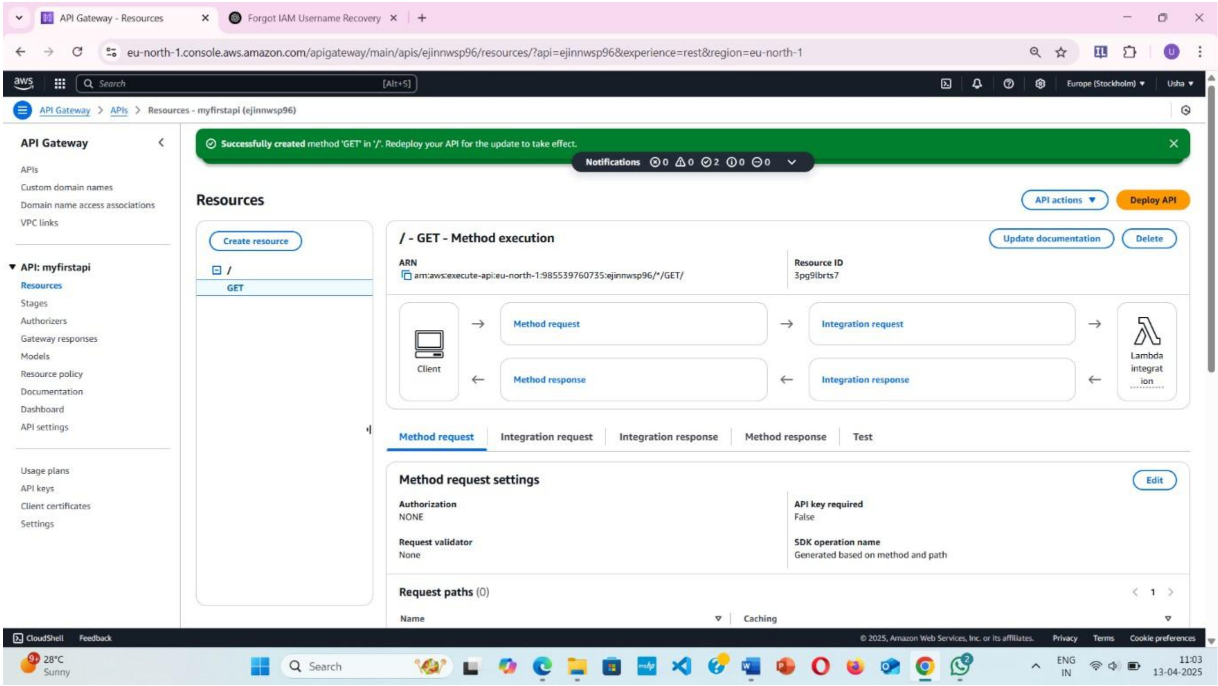
Create method And Select Lambda Integration

Select GET method



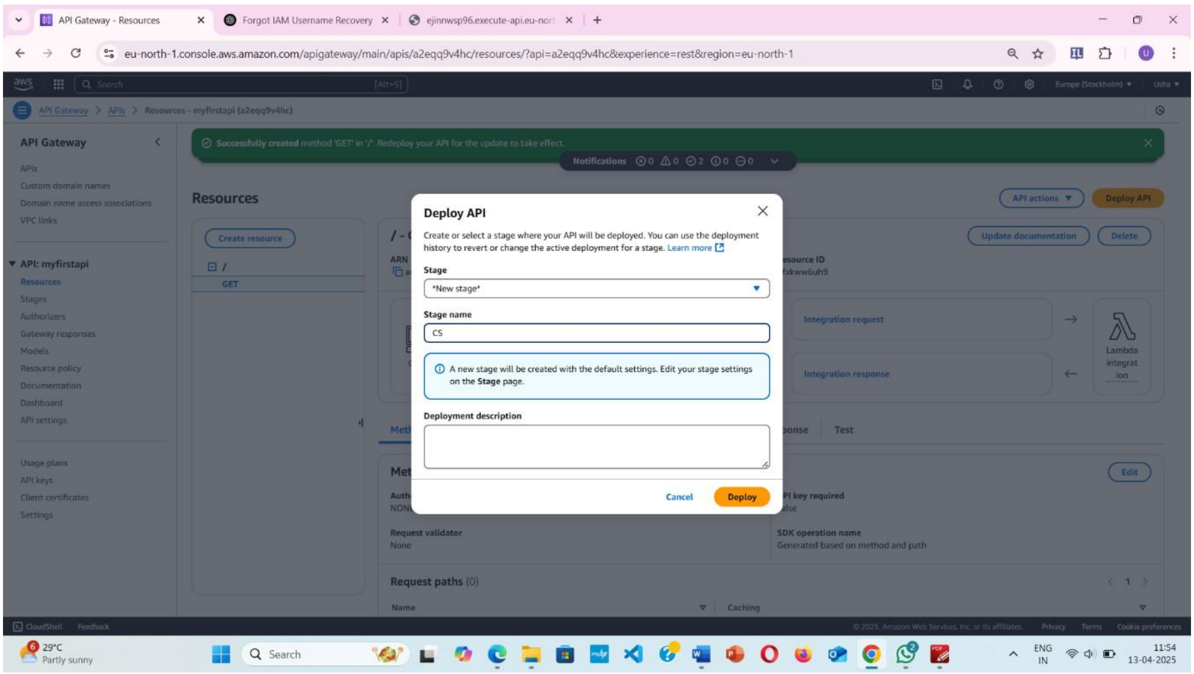


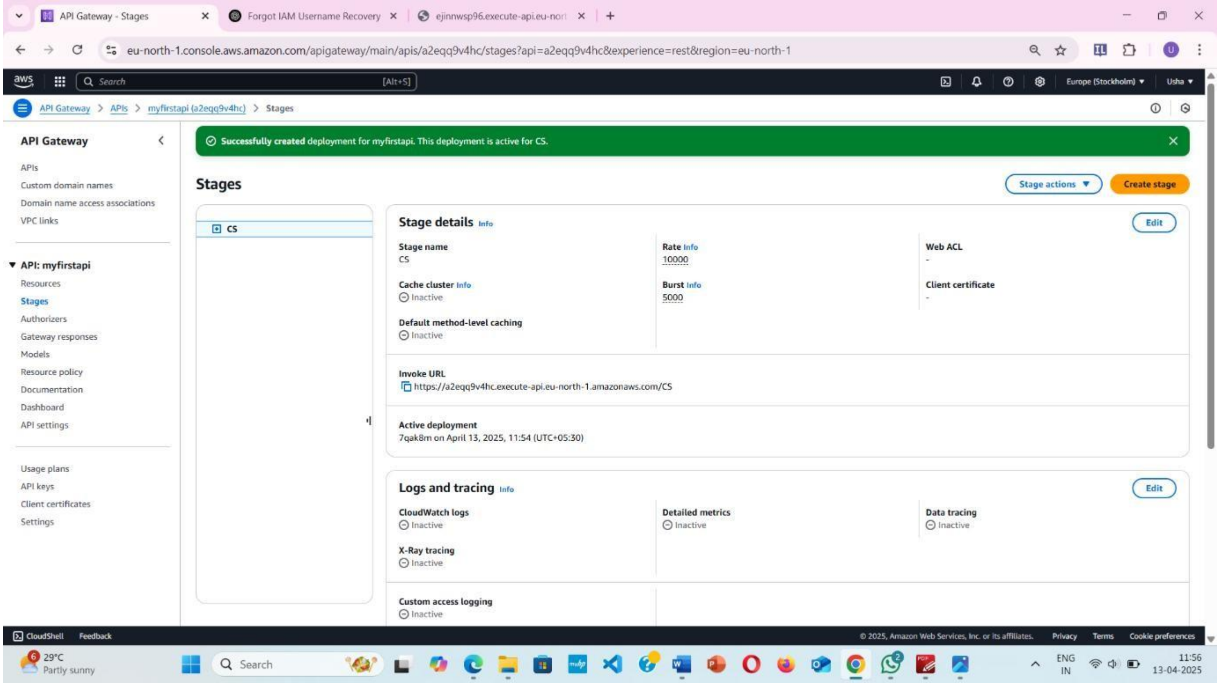
Successfully created method "GET"



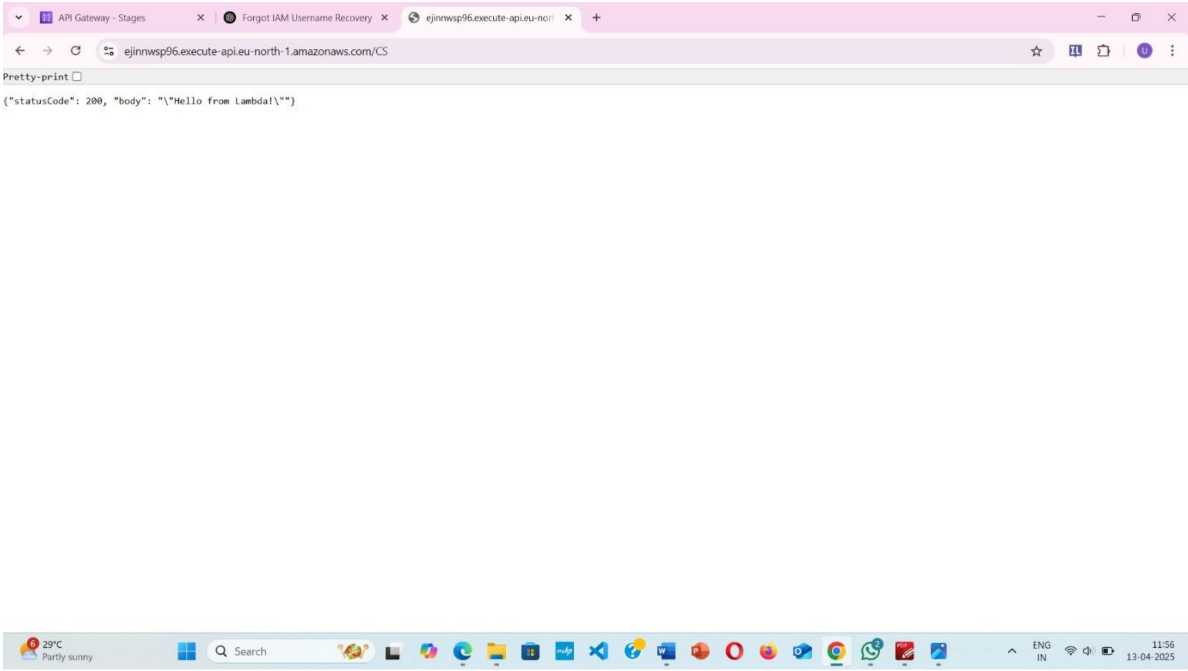
To Test API

Deploy API





Use URL and copy in new tab



Experiment No – 5

Creating Lambda Functions Using the AWS SDK for Python

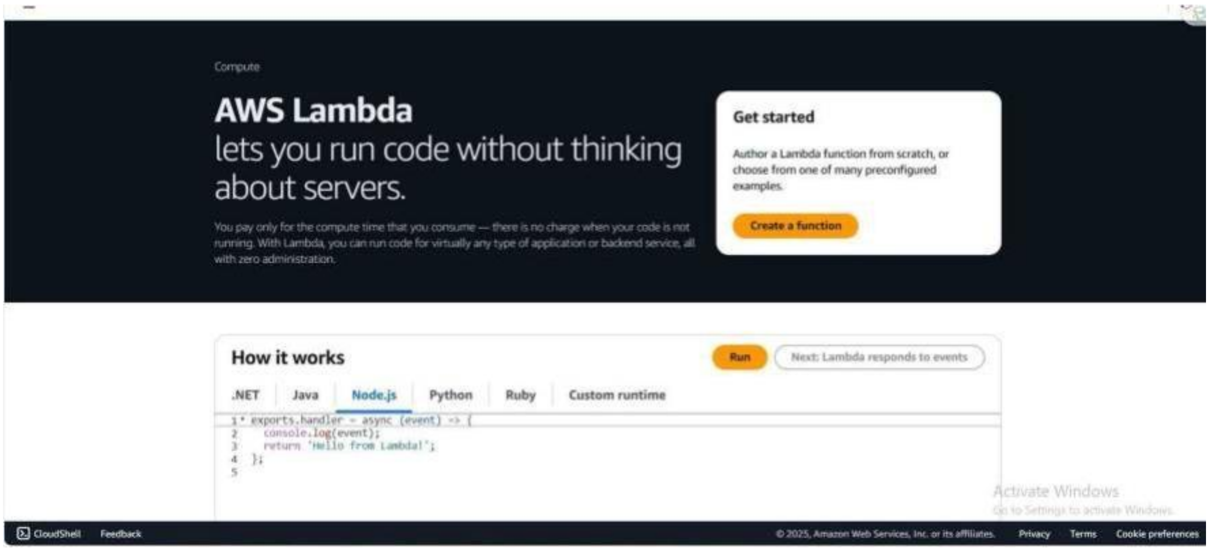
Objectives: Creating Lambda Functions Using the AWS SDK for Python

Apparatus Required: AWS Account

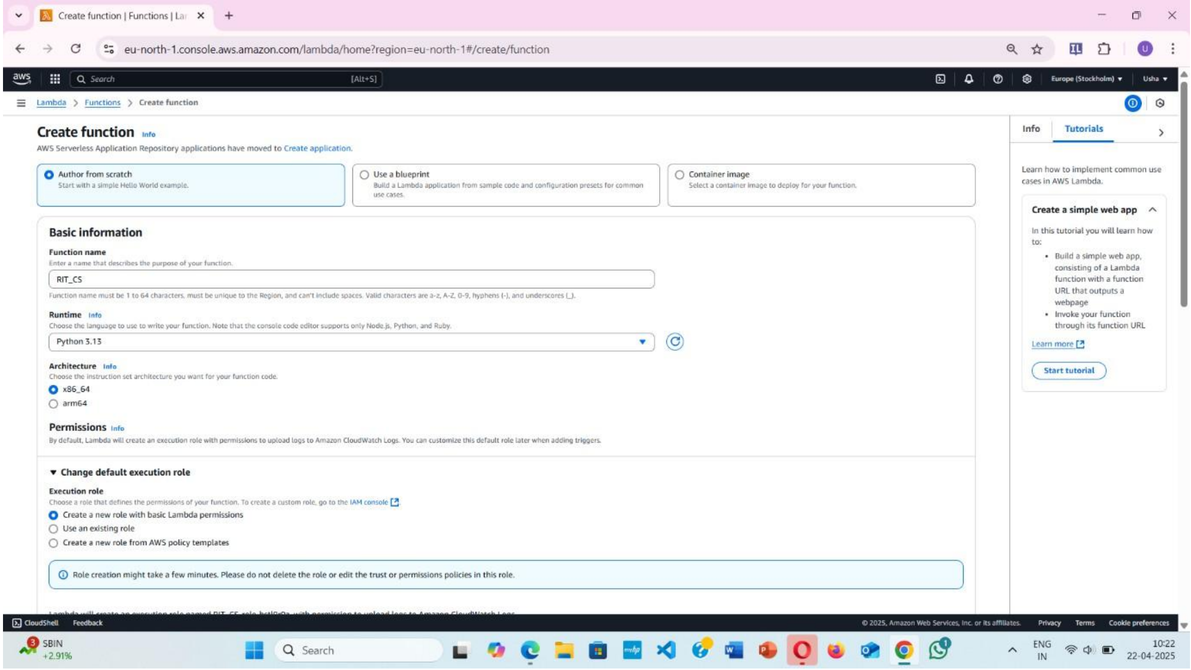
Pre-Requisite: AWS basic knowledge, basics of lambda functions and Python

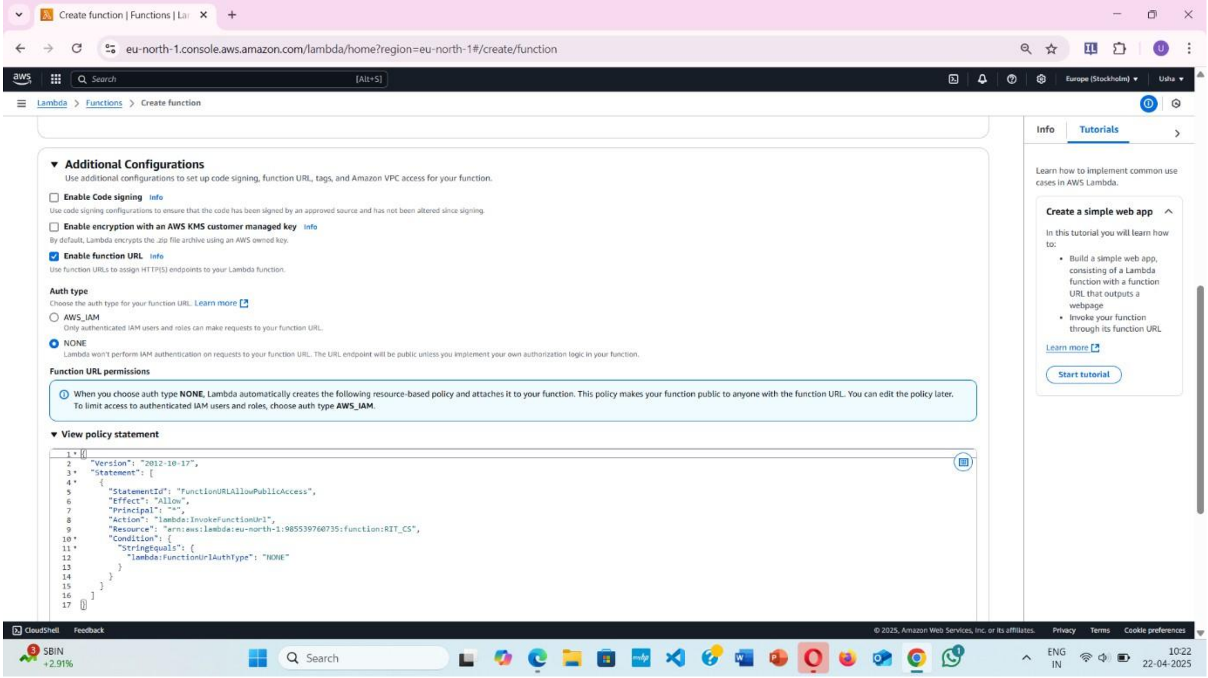
Introduction: An AWS Lambda function is a serverless function that runs code in response to events, such as user actions or system updates. You can use Lambda functions to create custom backend services, extend other AWS services, or trigger actions in other services.

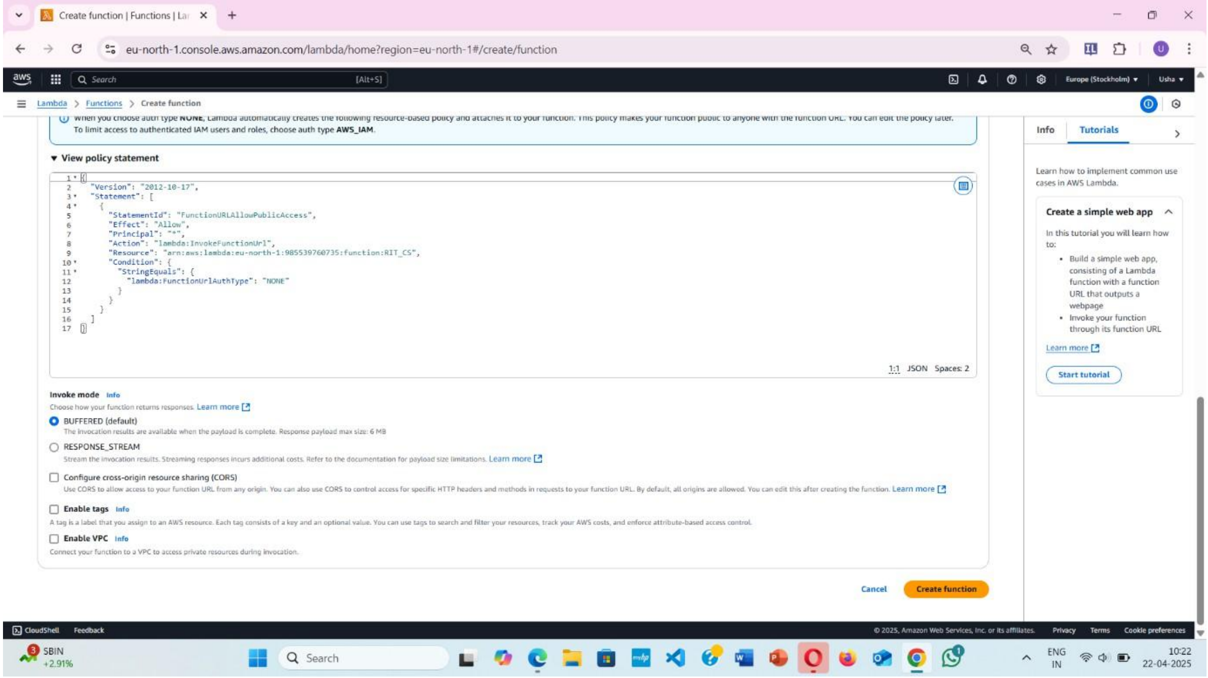
Procedure:

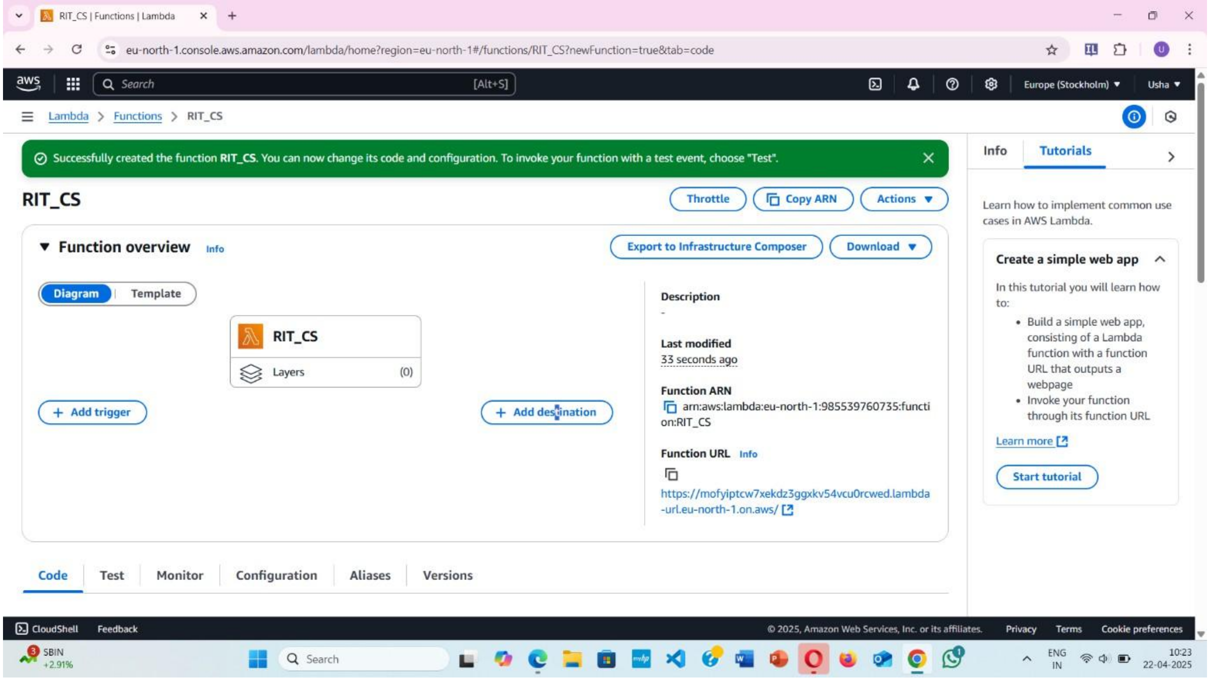


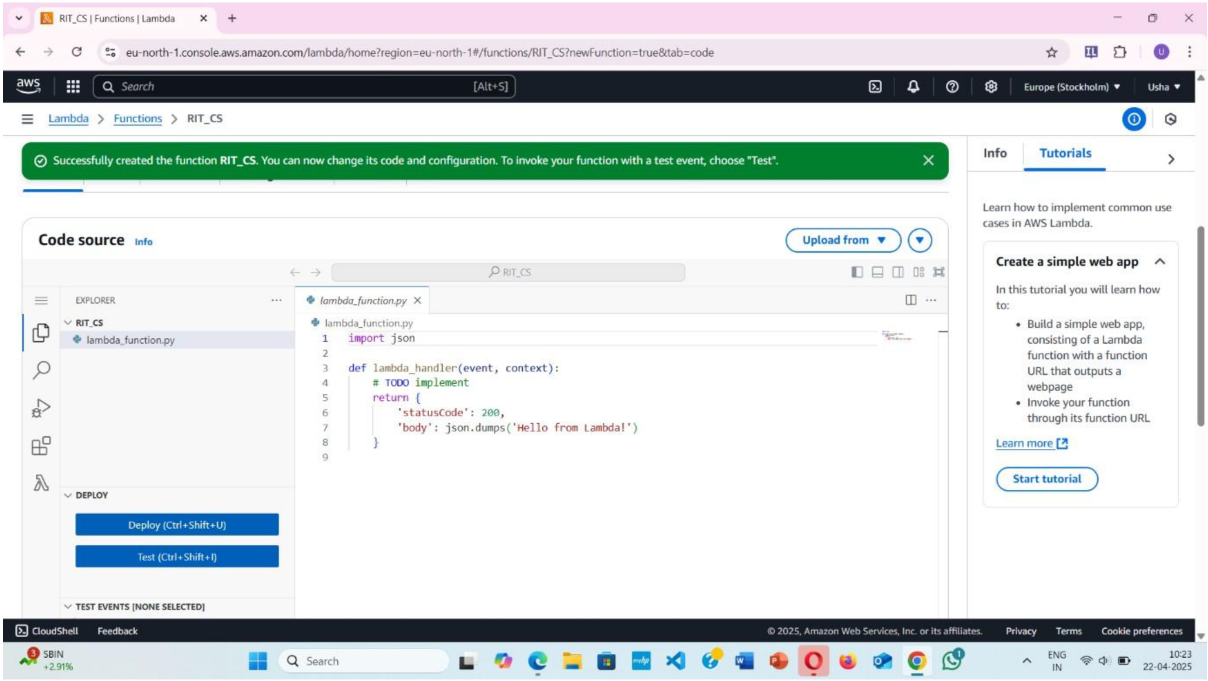
CREATE FUNCTION



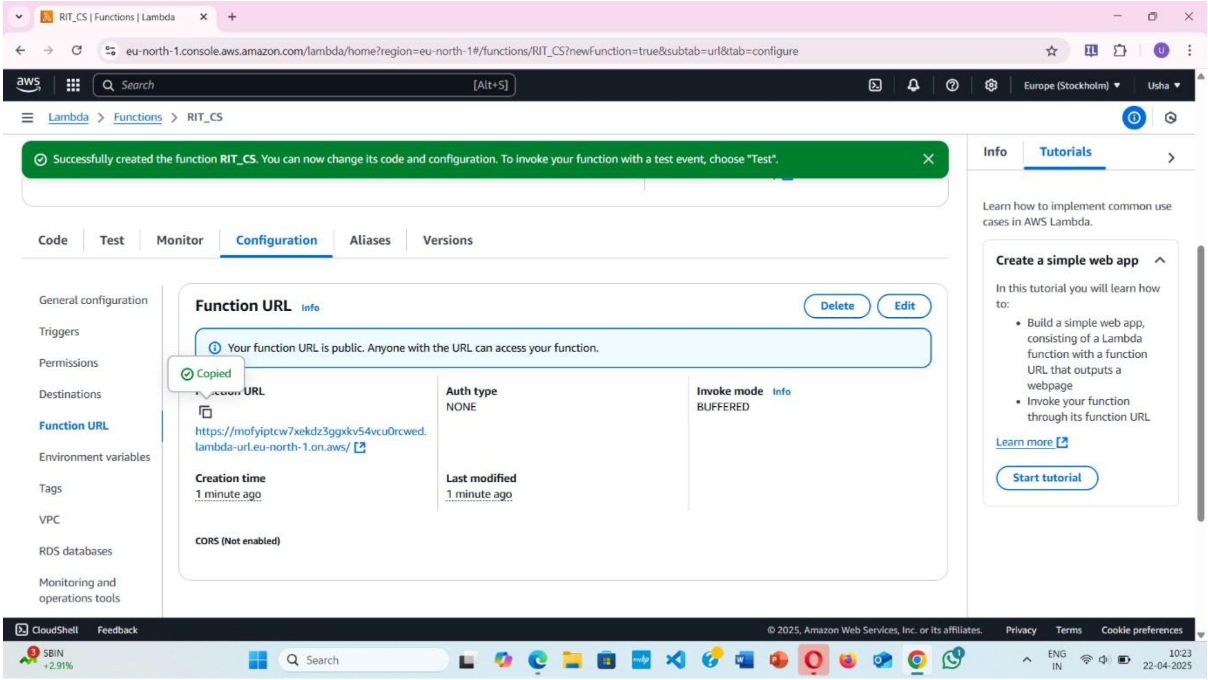


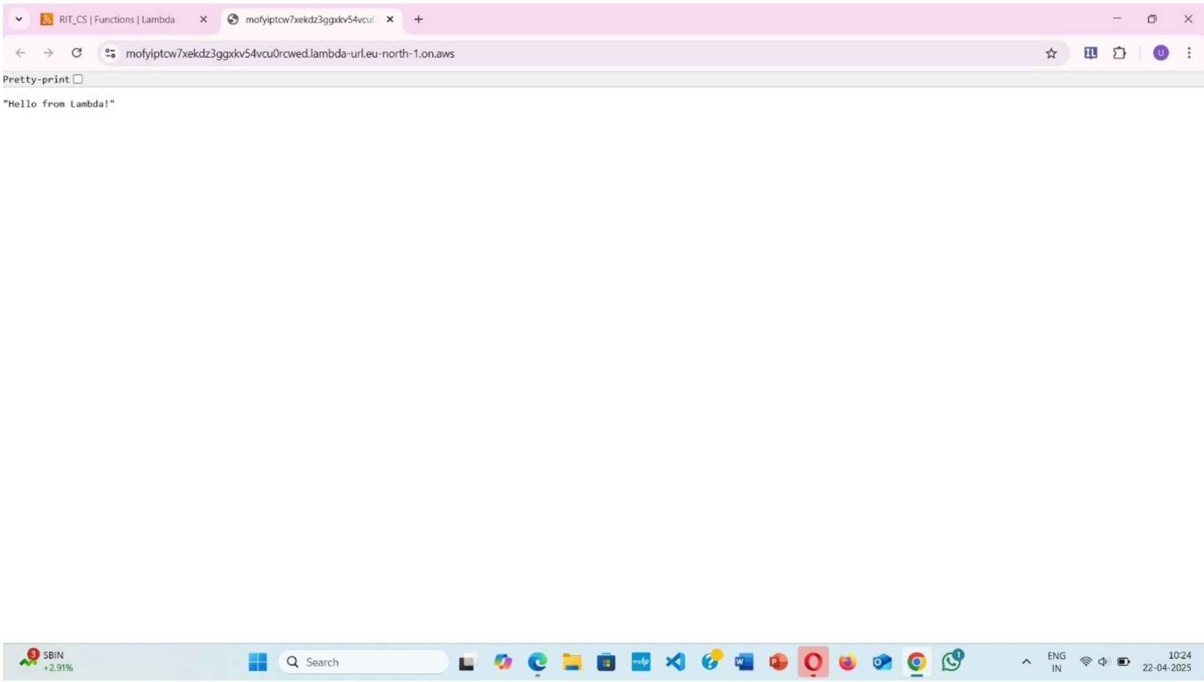



Successfully created the function RIT_CS. You can now change its code and configuration. To invoke your function with a test event, choose "Test".
RIT_CS
Function overview
Description
Last modified
33 seconds ago
Function ARN
arn:aws:lambda:eu-north-1:985539760735:function:RIT_CS
Function URL
https://mofyiptcw7xekdz3ggxkv54vcu0rcwed.lambda-url.eu-north-1.on.aws/
Code
Test
Monitor
Configuration
Aliases
Versions


Successfully created the function RIT_CS. You can now change its code and configuration. To invoke your function with a test event, choose "Test".
Code source
lambda_function.py
import json
def lambda_handler(event, context):
# TODO implement
return {
'statusCode': 200,
'body': json.dumps('Hello from Lambda!')
}
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)

CONFIGERATIONS →FUNCTION URL → CHECK URL RUNS





Experiment No – 06

Migrating a Web Application to Docker Containers

Objectives: Migrating a Web Application to Docker Containers

Apparatus Required: AWS Account

Pre-Requisite: AWS basics knowledge, Basics of docker

Introduction: Docker is a set of platform as a service (PaaS) products that use OS-level virtualization to deliver software in packages called containers.

Docker is a software platform that packages software into containers.

Docker images are read-only templates that contain instructions for creating a Container.

A Docker image is a snapshot or blueprint of the libraries and dependencies required inside a container for an application to run.

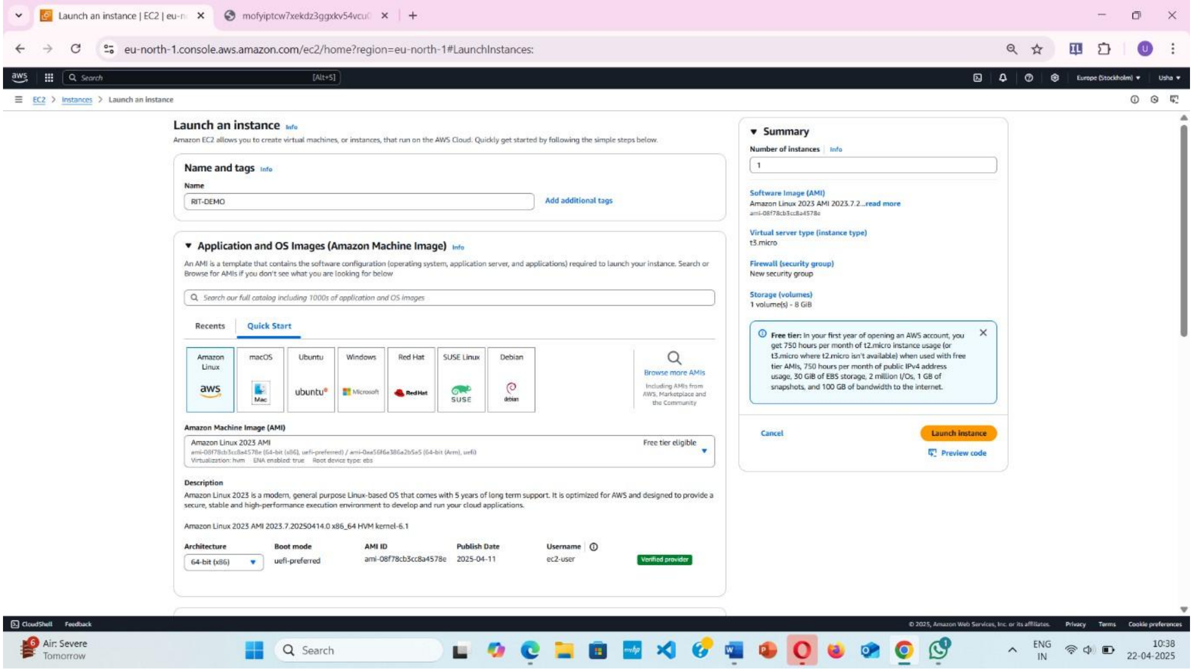
A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime,

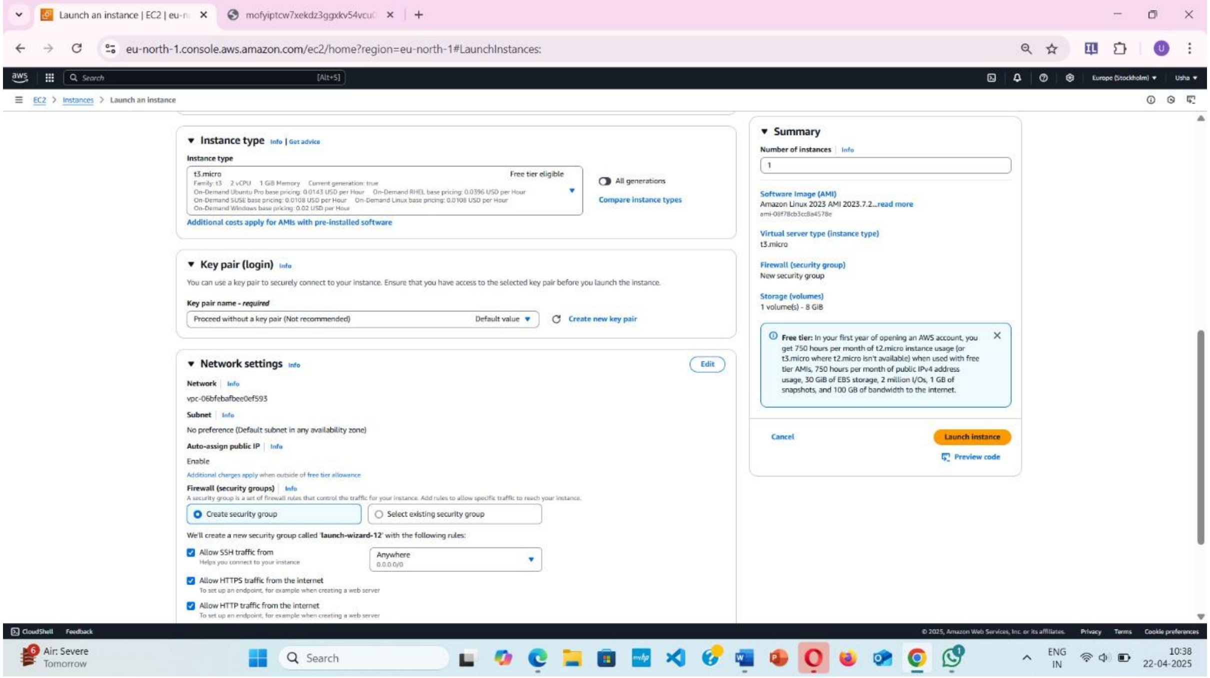
Procedure:

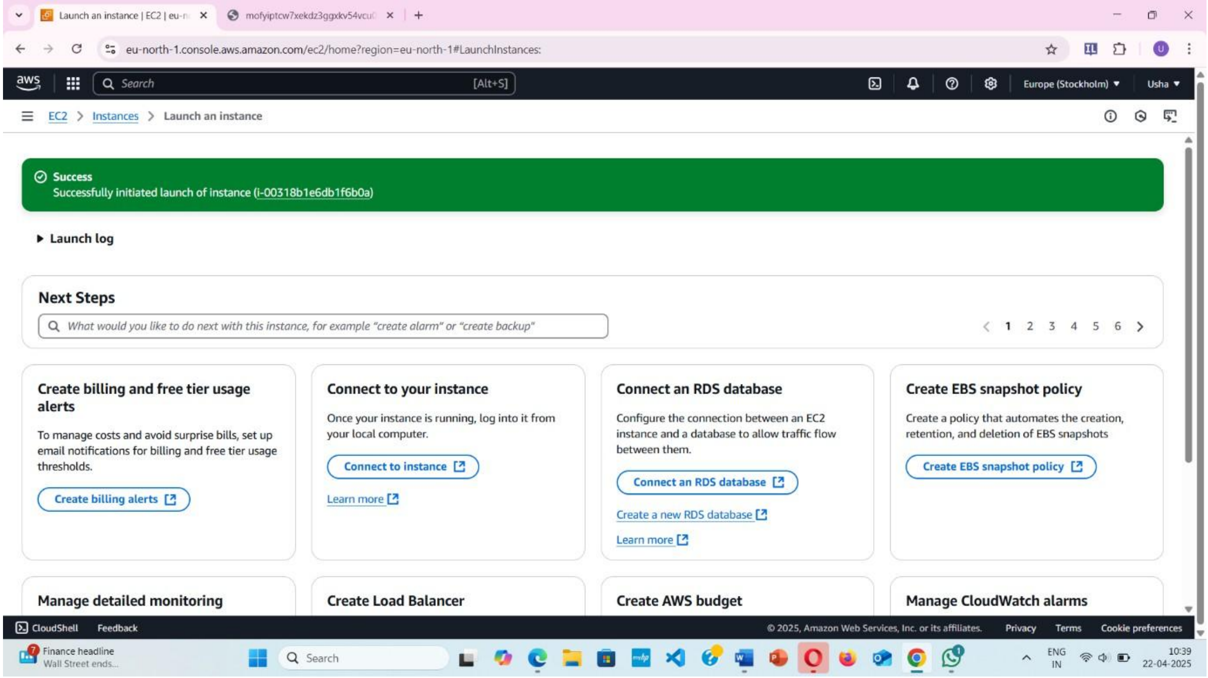
Create EC2 Instance

Linux Machine

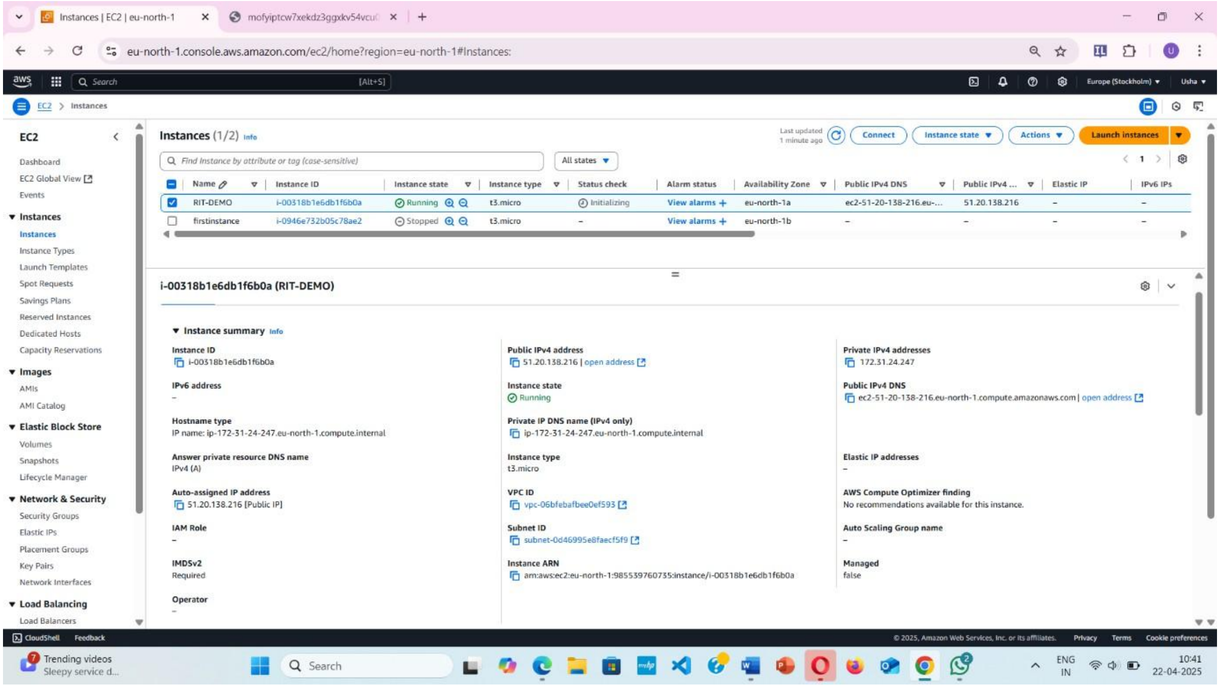
Enable HTTPS and H TTP



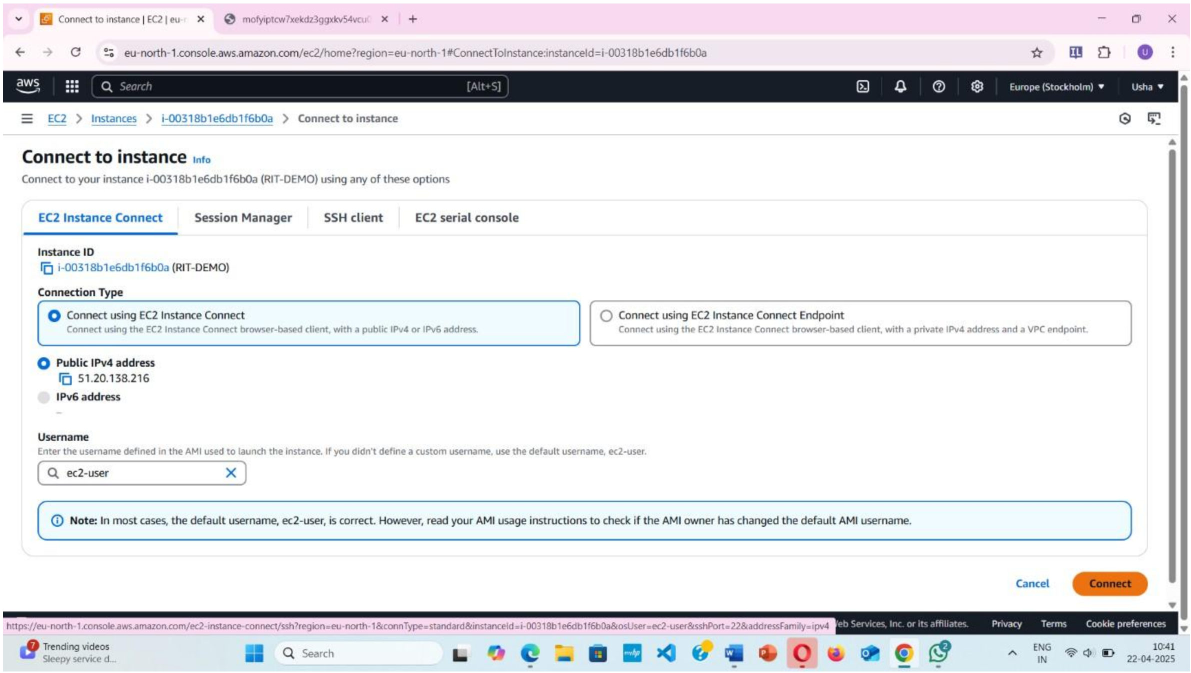

Instance type
Key pair (login)
Network settings
Summary
Launch instance


Success
Successfully initiated launch of instance (i-00318b1e6db1f6b0a)
Launch log
Next Steps
Create billing and free tier usage alerts
Connect to your instance
Connect an RDS database
Create EBS snapshot policy
Manage detailed monitoring
Create Load Balancer
Create AWS budget
Manage CloudWatch alarms

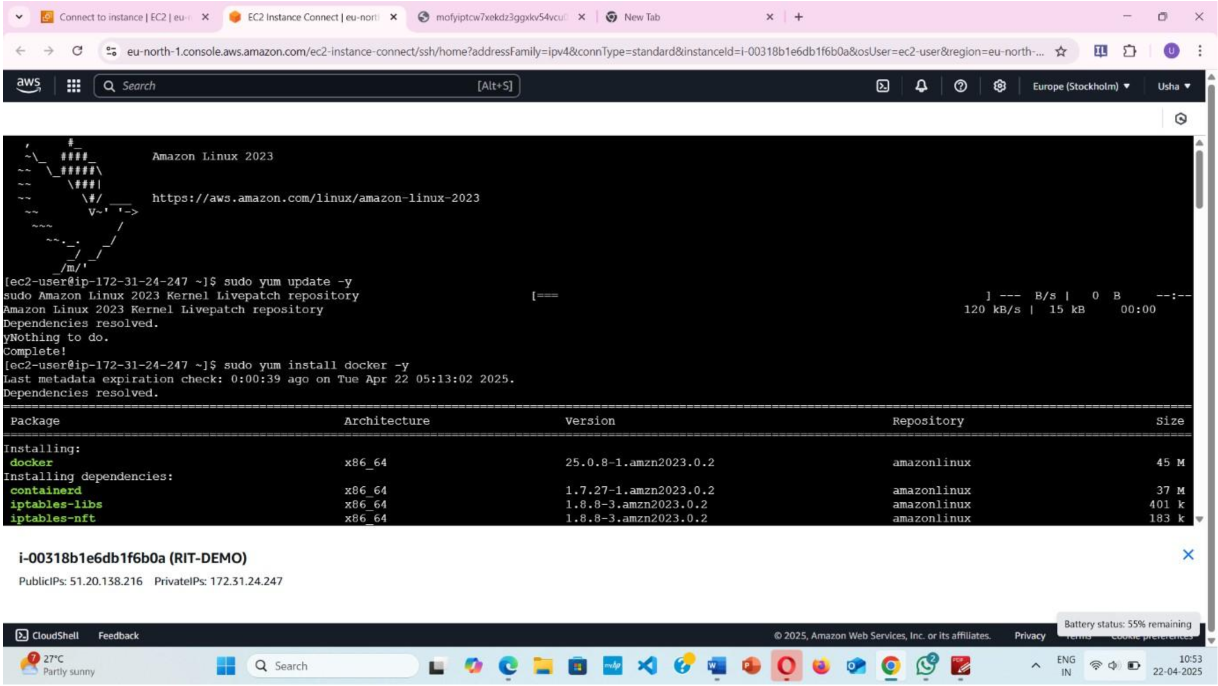
Go To Instance



Press connect



It will take you to Linux Machine



EXECUTE BELOW COMMANDS IN LINUX ENGINE

sudo yum update –y

sudo yum install docker –y

sudo service docker start

sudo service docker status

(ctrl Z)

sudo su

docker version

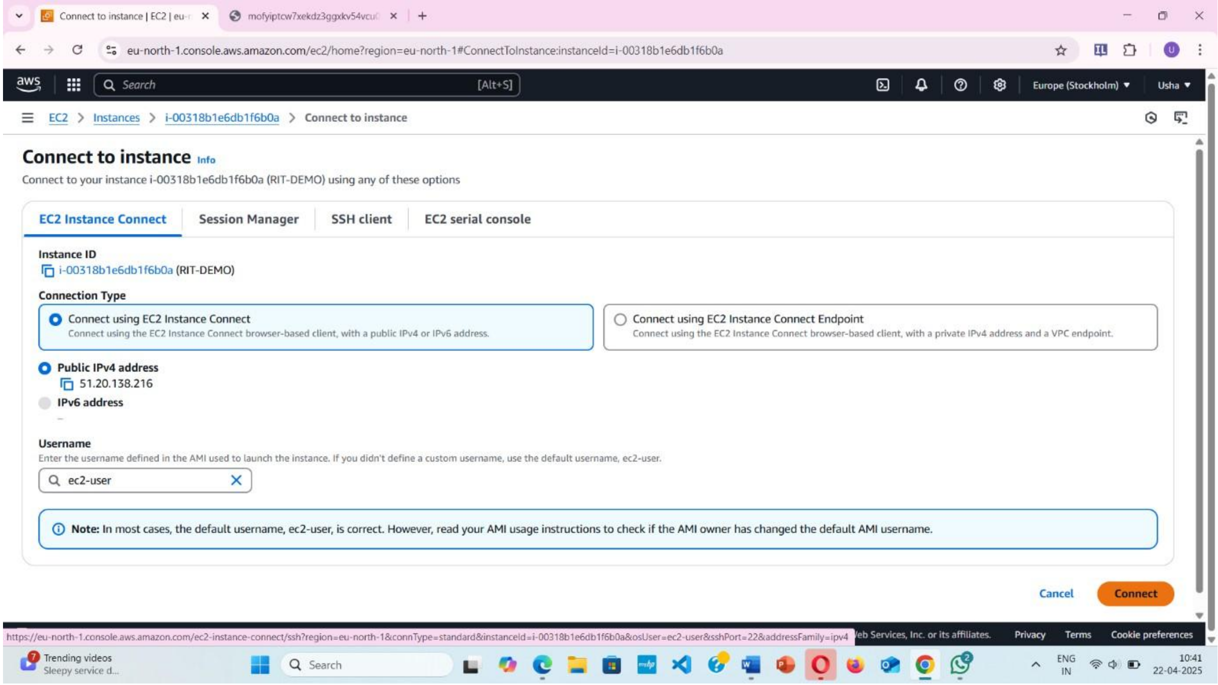
docker pull nginx

docker images

docker run -d -p 80:80 nginx

docker ps

after this go to ec2 instance → connect → copy public ip



Paste it in browser